not be acceptable to the environment, but that problem will be dealt with in subsequent sections.

If B is a type refinement of A, and A is a type refinement of B, then we say that A and B are **type equivalent**. They have the same input and output ports, and the types of the ports are the same.

Example 14.3: Let A represent the nondeterministic traffic light model in Figure 3.12 and B represent the more detailed deterministic model in Figure 3.10. The ports and their types are identical for both machines, so they are type equivalent. Hence, replacing A with B or vice versa in any environment will not cause type system problems.

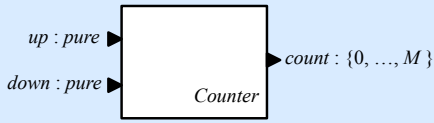
Notice that since Figure 3.12 ignores the *pedestrian* input, it might seem reasonable to omit that port. Let A' represent a variant of Figure 3.12 without the *pedestrian* input. It is not safe to replace A' with B in all environments, because B requires an input *pedestrian* signal, but A' can be used in an environment that provides no such input.

14.3 Language Equivalence and Containment

To replace a machine A with a machine B, looking at the data types of the inputs and outputs alone is usually not enough. If A is a specification and B is an implementation, then normally A imposes more constraints than just data types. If B is an optimization of A (e.g., a lower cost implementation or a refinement that adds functionality or leverages new technology), then B normally needs to conform in some way with the functionality of A.

In this section, we consider a stronger form of equivalence and refinement. Specifically, equivalence will mean that given a particular sequence of input valuations, the two machines produce the same output valuations.

Example 14.4: The garage counter of Figure 3.4, discussed in Example 3.4, is type equivalent to the extended state machine version in Figure 3.8. The actor model is shown below:



However, these two machines are equivalent in a much stronger sense than simply type equivalence. These two machines behave in exactly the same way, as viewed from the outside. Given the same input sequence, the two machines will produce the same output sequence.

Consider a port p of a state machine with type V_p. This port will have a sequence of values from the set $V_p \cup \{absent\}$, one value at each reaction. We can represent this sequence as a function of the form

$$s_p : \mathbb{N} \to V_p \cup \{absent\}.$$

This is the signal received on that port (if it is an input) or produced on that port (if it is an output). Recall that a behavior of a state machine is an assignment of such a signal to each port of such a machine. Recall further that the language $L(M)$ of a state machine M is the set of all behaviors for that state machine. Two machines are said to be **language equivalent** if they have the same language.

Example 14.5: A behavior of the garage counter is a sequence of *present* and *absent* valuations for the two inputs, *up* and *down*, paired with the corresponding output sequence at the output port, *count*. A specific example is given in Example 3.16. This is a behavior of both Figures 3.4 and 3.8. All behaviors of Figure 3.4 are also behaviors of 3.8 and vice versa. These two machines are language equivalent.

In the case of a nondeterministic machine M, two distinct behaviors may share the same input signals. That is, given an input signal, there is more than one possible output se-

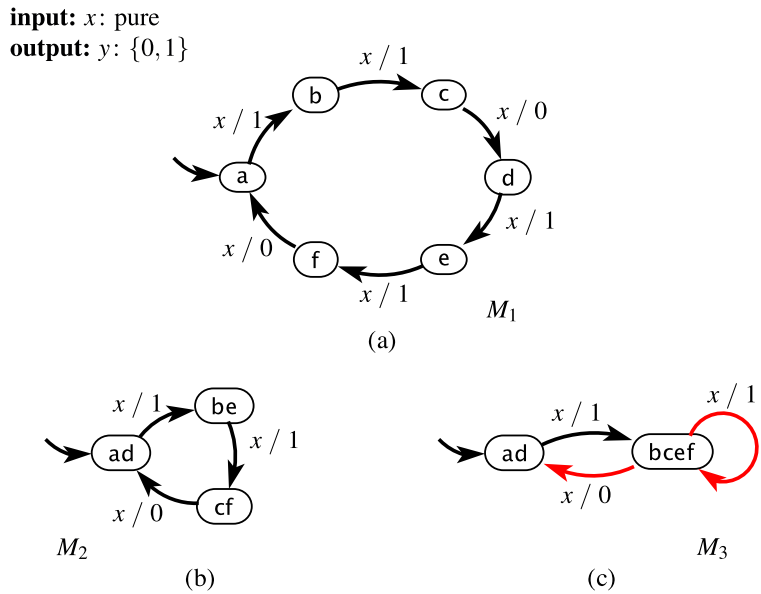


Figure 14.2: Three state machines where (a) and (b) have the same language, and that language is contained by that of (c).

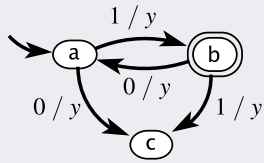
quence. The language $L(M)$ includes all possible behaviors. Just like deterministic machines, two nondeterministic machines are language equivalent if they have the same language.

Suppose that for two state machines A and B, $L(A) \subset L(B)$. That is, B has behaviors that A does not have. This is called **language containment**. A is said to be a **language refinement** of B. Just as with type refinement, language refinement makes an assertion about the suitability of A as a replacement for B. If every behavior of B is acceptable to an environment, then every behavior of A will also be acceptable to that environment. A can substitute for B.

Finite Sequences and Accepting States

A complete execution of the FSMs considered in this text is infinite. Suppose that we are interested in only the finite executions. To do this, we introduce the notion of an **accepting state**, indicated with a double outline as in state b in the example below:

input: x: $\{0, 1\}$
output: y: pure



Let $L_a(M)$ denote the subset of the language $L(M)$ that results from executions that terminate in an accepting state. Equivalently, $L_a(M)$ includes only those behaviors in $L(M)$ with an infinite tail of stuttering reactions that remain in an accepting state. All such executions are effectively finite, since after a finite number of reactions, the inputs and outputs will henceforth be *absent*, or in LTL, $\mathbf{FG}\neg p$ for every port p.

We call $L_a(M)$ the **language accepted by an FSM** M. A behavior in $L_a(M)$ specifies for each port p a finite **string**, or a finite sequence of values from the type V_p. For the above example, the input strings (1), $(1, 0, 1)$, $(1, 0, 1, 0, 1)$, etc., are all in $L_a(M)$. So are versions of these with an arbitrary finite number of *absent* values between any two present values. When there is no ambiguity, we can write these strings 1, 101, 10101, etc.

In the above example, in all behaviors in $L_a(M)$, the output is present a finite number of times, in the same reactions when the input is present.

The state machines in this text are receptive, meaning that at each reaction, each input port p can have any value in its type V_p or be *absent*. Hence, the language $L(M)$ of the machine above includes all possible sequences of input valuations. $L_a(M)$ excludes any of these that do not leave the machine in an accepting state. For example, any input sequence with two 1's in a row and the infinite sequence $(1, 0, 1, 0, \cdots)$ are in $L(M)$ but not in $L_a(M)$.

Note that it is sometimes useful to consider language containment when referring to the language *accepted* by the state machine, rather than the language that gives all behaviors of the state machine.

Accepting states are also called **final state**s, since for any behavior in $L_a(M)$, it is the last state of the machine. Accepting states are further explored in Exercise 2.

Regular Languages and Regular Expressions

A **language** is a set of sequences of values from some set called its **alphabet**. A language accepted by an FSM is called a **regular language**. A classic example of a language that is not regular has sequences of the form 0^n1^n, a sequence of n zeros followed by n ones. It is easy to see that no *finite* state machine can accept this language because the machine would have to count the zeros to ensure that the number of ones matches. And the number of zeros is not bounded. On the other hand, the input sequences accepted by the FSM in the box on page 384, which have the form $10101\cdots01$, are regular.

A **regular expression** is a notation for describing regular languages. A central feature of regular expressions is the **Kleene star** (or **Kleene closure**), named after the American mathematician Stephen Kleene (who pronounced his name KLAY-nee). The notation $V*$, where V is a set, means the set of all finite sequences of elements from V. For example, if $V = \{0, 1\}$, then $V*$ is a set that includes the **empty sequence** (often written λ), and every finite sequence of zeros and ones.

The Kleene star may be applied to sets of sequences. For example, if $A = \{00, 11\}$, then $A*$ is the set of all finite sequences where zeros and ones always appear in pairs. In the notation of regular expressions, this is written `(00|11)*`, where the vertical bar means "or." What is inside the parentheses defines the set A.

Regular expressions are sequences of symbols from an alphabet and sets of sequences. Suppose our alphabet is $A = \{\texttt{a}, \texttt{b}, \cdots, \texttt{z}\}$, the set of lower-case characters. Then `grey` is a regular expression denoting a single sequence of four characters. The expression `grey|gray` denotes a set of two sequences. Parentheses can be used to group sequences or sets of sequences. For example, `(grey)|(gray)` and `gr(e|a)y` mean the same thing.

Regular expressions also provide convenience notations to make them more compact and readable. For example, the + operator means "one or more," in contrast to the Kleene star, which means "zero or more." For example, `a+` specifies the sequences `a`, `aa`, `aaa`, etc.; it is the same as `a(a*)`. The `?` operator species "zero or one." For example, `colou?r` specifies a set with two sequences, `color` and `colour`; it is the same as `colo(λ|u)r`, where λ denotes the empty sequence.

Regular expressions are commonly used in software systems for pattern matching. A typical implementation provides many more convenience notations than the ones illustrated here.

Example 14.6: Machines M_1 and M_2 in Figure 14.2 are language equivalent. Both machines produce output $1, 1, 0, 1, 1, 0, \cdots$, possibly interspersed with *absent* if the input is absent in some reactions.

Machine M_3, however, has more behaviors. It can produce any output sequence that M_1 and M_2 can produce, but it can also produce other outputs given the same inputs. Thus, M_1 and M_2 are both language refinements of M_3.

Language containment assures that an abstraction is sound with respect to LTL formulas about input and output sequences. That is, if A is a language refinement of B, then any LTL formula about inputs and outputs that holds for B also holds for A.

Example 14.7: Consider again the machines in Figure 14.2. M_3 might be a specification. For example, if we require that any two output values 0 have at

Probing Further: Omega Regular Languages

The regular languages discussed in the boxes on pages 384 and 385 contain only finite sequences. But embedded systems most commonly have infinite executions. To extend the idea of regular languages to infinite runs, we can use a **Büchi automaton**, named after Julius Richard Büchi, a Swiss logician and mathematician. A Büchi automaton is a possibly nondeterministic FSM that has one or more accepting states. The language accepted by the FSM is defined to be the set of behaviors that visit one or more of the accepting states infinitely often; in other words, these behaviors satisfy the LTL formula $\mathbf{GF}(s_1 \vee \cdots \vee s_n)$, where $s_1, \cdots, s_n$ are the accepting states. Such a language is called an **omega-regular language** or **ω-regular language**, a generalization of regular languages. The reason for using ω in the name is because ω is used to construct infinite sequences, as explained in the box on page 500.

As we will see in Chapter 15, many model checking questions can be expressed by giving a Büchi automaton and then checking to see whether the ω-regular language it defines contains any sequences.

least one intervening 1, then M_3 is a suitable specification of this requirement. This requirement can be written as an LTL formula as follows:

$$\mathbf{G}((y = 0) \Rightarrow \mathbf{X}((y \neq 0)\mathbf{U}(y = 1))).$$

If we prove that this property holds for M_3, then we have implicitly proved that it also holds for M_1 and M_2.

We will see in the next section that language containment is *not sound* with respect to LTL formulas that refer to states of the state machines. In fact, language containment does not require the state machines to have the same states, so an LTL formula that refers to the states of one machine may not even apply to the other machine. A sound abstraction that references states will require simulation.

Language containment is sometimes called **trace containment**, but here the term "trace" refers only to the observable trace, not to the execution trace. As we will see next, things get much more subtle when considering execution traces.

14.4 Simulation

Two nondeterministic FSMs may be language equivalent but still have observable differences in behavior in some environments. Language equivalence merely states that given the same sequences of input valuations, the two machines are *capable* of producing the same sequences of output valuations. However, as they execute, they make choices allowed by the nondeterminism. Without being able to see into the future, these choices could result in one of the machines getting into a state where it can no longer match the outputs of the other.

When faced with a nondeterministic choice, each machine is free to use any policy to make that choice. Assume that the machine cannot see into the future; that is, it cannot anticipate future inputs, and it cannot anticipate future choices that any other machine will make. For two machines to be equivalent, we will require that each machine be able to make choices that allow it to match the reaction of the other machine (producing the same outputs), and further allow it to continue to do such matching in the future. It turns out that language equivalence is not strong enough to ensure that this is possible.

Example 14.8: Consider the two state machines in Figure 14.3. Suppose that M_2 is acceptable in some environment (every behavior it can exhibit in that environment is consistent with some specification or design intent). Is it safe for M_1 to replace M_2? The two machines are language equivalent. In all behaviors, the output is one of two finite strings, 01 or 00, for both machines. So it would seem that M_1 can replace M_2. But this is not necessarily the case.

Suppose we compose each of the two machines with its own copy of the environment that finds M_2 acceptable. In the first reaction where x is *present*, M_1 has no choice but to take the transition to state b and produce the output $y = 0$. However, M_2 must choose between f and h. Whichever choice it makes, M_2 matches the output $y = 0$ of M_1 but enters a state where it is no longer able to always match the outputs of M_1. If M_1 can observe the state of M_2 when making its choice, then in the second reaction where x is *present*, it can choose a transition that M_2 can *never* match. Such a policy for M_1 ensures that the behavior of M_1, given the same inputs, is never the same as the behavior of M_2. Hence, it is not safe to replace M_2 with M_1.

On the other hand, if M_1 is acceptable in some environment, is it safe for M_2 to replace M_1? What it means for M_1 to be acceptable in the environment is that whatever decisions it makes are acceptable. Thus, in the second reaction where x is *present*, both outputs $y = 1$ and $y = 0$ are acceptable. In this second reaction, M_2 has no choice but to produce one or the other these outputs, and it will inevitably transition to a state where it continues to match the outputs of M_1 (henceforth forever *absent*). Hence it is safe for M_2 to replace M_1.

In the above example, we can think of the machines as maliciously trying to make M_1 look different from M_2. Since they are free to use any policy to make choices, they are free to use policies that are contrary to our goal to replace M_2 with M_1. Note that the machines do not need to know the future; it is sufficient to simply have good visibility of the present. The question that we address in this section is: under what circumstances can we assure that there is no policy for making nondeterministic choices that can make machine M_1 observably different from M_2? The answer is a stronger form of equivalence called bisimulation and a refinement relation called simulation. We begin with the simulation relation.

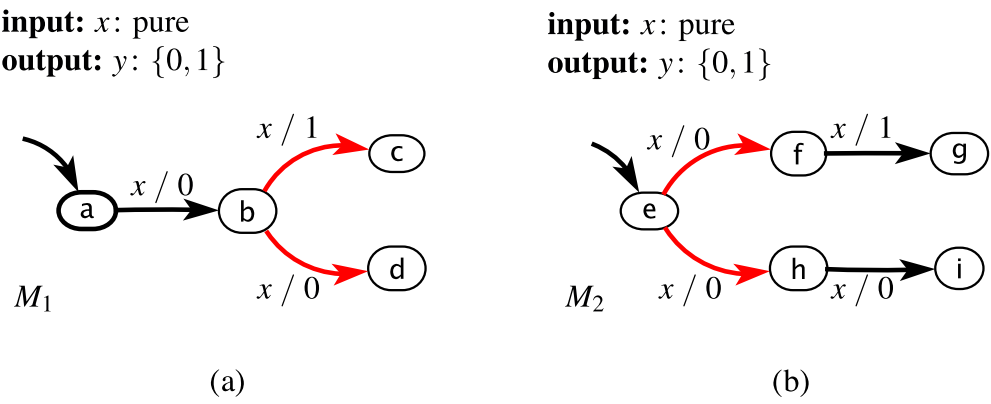


Figure 14.3: Two state machines that are language equivalent but where M_2 does not simulate M_1 (M_1 does simulate M_2).

14.4.1 Simulation Relations

First, notice that the situation given in Example 14.8 is not symmetric. It is safe for M_2 to replace M_1, but not the other way around. Hence, M_2 is a refinement of M_1, in a sense that we will now establish. M_1, on the other hand, is not a refinement of M_2.

The particular kind of refinement we now consider is a **simulation refinement**. The following statements are all equivalent:

- M_2 is a simulation refinement of M_1.
- M_1 simulates M_2.
- M_1 is a simulation abstraction of M_2.

Simulation is defined by a **matching game**. To determine whether M_1 simulates M_2, we play a game where M_2 gets to move first in each round. The game starts with both machines in their initial states. M_2 moves first by reacting to an input valuation. If this involves a nondeterministic choice, then it is allowed to make any choice. Whatever it choses, an output valuation results and M_2's turn is over.

It is now M_1's turn to move. It must react to the same input valuation that M_2 reacted to. If this involves a nondeterministic choice, then it must make a choice that matches the output valuation of M_2. If there are multiple such choices, it must select one without knowledge of the future inputs or future moves of M_2. Its strategy should be to choose one that enables it to continue to match M_2, regardless of what future inputs arrive or future decisions M_2 makes.

Machine M_1 "wins" this matching game (M_1 simulates M_2) if it can always match the output symbol of machine M_2 for all possible input sequences. If in any reaction M_2 can produce an output symbol that M_1 cannot match, then M_1 does not simulate M_2.

Example 14.9: In Figure 14.3, M_1 simulates M_2 but not vice versa. To see this, first play the game with M_2 moving first in each round. M_1 will always be able to match M_2. Then play the game with M_1 moving first in each round. M_2 will not always be able to match M_1. This is true even though the two machines are language equivalent.

Interestingly, if M_1 simulates M_2, it is possible to compactly record all possible games over all possible inputs. Let S_1 be the states of M_1 and S_2 be the states of M_2. Then a simulation relation $S \subseteq S_2 \times S_1$ is a set of pairs of states occupied by the two machines in each round of the game for all possible inputs. This set summarizes all possible plays of the game.

Example 14.10: In Figure 14.3,

$$S_1 = \{\mathsf{a}, \mathsf{b}, \mathsf{c}, \mathsf{d}\}$$

and

$$S_2 = \{\mathsf{e}, \mathsf{f}, \mathsf{g}, \mathsf{h}, \mathsf{i}\}.$$

The simulation relation showing that M_1 simulates M_2 is

$$S = \{(\mathsf{e}, \mathsf{a}), (\mathsf{f}, \mathsf{b}), (\mathsf{h}, \mathsf{b}), (\mathsf{g}, \mathsf{c}), (\mathsf{i}, \mathsf{d})\}$$

First notice that the pair (e, a) of initial states is in the relation, so the relation includes the state of the two machines in the first round. In the second round, M_2 may be in either f or h, and M_1 will be in b. These two possibilities are also accounted for. In the third round and beyond, M_2 will be in either g or i, and M_1 will be in c or d.

There is no simulation relation showing that M_2 simulates M_1, because it does not.

A simulation relation is complete if it includes all possible plays of the game. It must therefore account for all reachable states of M_2, the machine that moves first, because M_2's moves are unconstrained. Since M_1's moves are constrained by the need to match M_2, it is not necessary to account for all of its reachable states.

14.4.2 Formal Model

Using the formal model of nondeterministic FSMs given in Section 3.5.1, we can formally define a simulation relation. Let

$$M_1 = (States_1, Inputs, Outputs, possibleUpdates_1, initialState_1),$$

and

$$M_2 = (States_2, Inputs, Outputs, possibleUpdates_2, initialState_2).$$

Assume the two machines are type equivalent. If either machine is deterministic, then its *possibleUpdates* function always returns a set with only one element in it. If M_1 simulates M_2, the simulation relation is given as a subset of $States_2 \times States_1$. Note the ordering here; the machine that moves first in the game, M_2, the one being simulated, is first in $States_2 \times States_1$.

To consider the reverse scenario, if M_2 simulates M_1, then the relation is given as a subset of $States_1 \times States_2$. In this version of the game M_1 must move first.

We can state the "winning" strategy mathematically. We say that M_1 **simulates** M_2 if there is a subset $S \subseteq States_2 \times States_1$ such that

1. $(initialState_2, initialState_1) \in S$, and

2. If $(s_2, s_1) \in S$, then $\forall\, x \in \mathit{Inputs}$, and
   $\forall\, (s_2', y_2) \in \mathit{possibleUpdates}_2(s_2, x)$,
   there is a $(s_1', y_1) \in \mathit{possibleUpdates}_1(s_1, x)$ such that:

   (a) $(s_2', s_1') \in S$, and

   (b) $y_2 = y_1$.

This set S, if it exists, is called the **simulation relation**. It establishes a correspondence between states in the two machines. If it does not exist, then M_1 does not simulate M_2.

14.4.3 Transitivity

Simulation is **transitive**, meaning that if M_1 simulates M_2 and M_2 simulates M_3, then M_1 simulates M_3. In particular, if we are given simulation relations $S_{2,1} \subseteq \mathit{States}_2 \times \mathit{States}_1$ (M_1 simulates M_2) and $S_{3,2} \subseteq \mathit{States}_3 \times \mathit{States}_2$ (M_2 simulates M_3), then

$$S_{3,1} = \{(s_3, s_1) \in \mathit{States}_3 \times \mathit{States}_1 \mid \text{there exists } s_2 \in \mathit{States}_2 \text{ where } (s_3, s_2) \in S_{3,2} \text{ and } (s_2, s_1) \in S_{2,1}\}$$

Example 14.11: For the machines in Figure 14.2, it is easy to show that (c) simulates (b) and that (b) simulates (a). Specifically, the simulation relations are

$$S_{a,b} = \{(\mathsf{a}, \mathsf{ad}), (\mathsf{b}, \mathsf{be}), (\mathsf{c}, \mathsf{cf}), (\mathsf{d}, \mathsf{ad}), (\mathsf{e}, \mathsf{be}), (\mathsf{f}, \mathsf{cf})\}.$$

and

$$S_{b,c} = \{(\mathsf{ad}, \mathsf{ad}), (\mathsf{be}, \mathsf{bcef}), (\mathsf{cf}, \mathsf{bcef})\}.$$

By transitivity, we can conclude that (c) simulates (a), and that the simulation relation is

$$S_{a,c} = \{(\mathsf{a}, \mathsf{ad}), (\mathsf{b}, \mathsf{bcef}), (\mathsf{c}, \mathsf{bcef}), (\mathsf{d}, \mathsf{ad}), (\mathsf{e}, \mathsf{bcef}), (\mathsf{f}, \mathsf{bcef})\},$$

which further supports the suggestive choices of state names.

14.4.4 Non-Uniqueness of Simulation Relations

When a machine M_1 simulates another machine M_2, there may be more than one simulation relation.

Example 14.12: In Figure 14.4, it is easy to check that M_1 simulates M_2. Note that M_1 is nondeterministic, and in two of its states it has two distinct ways of matching the moves of M_2. It can arbitrarily choose from among these possibilities to match the moves. If from state b it always chooses to return to state a, then the simulation relation is

$$S_{2,1} = \{(\mathsf{ac}, \mathsf{a}), (\mathsf{bd}, \mathsf{b})\}.$$

Otherwise, if from state c it always chooses to return to state b, then the simulation relation is

$$S_{2,1} = \{(\mathsf{ac}, \mathsf{a}), (\mathsf{bd}, \mathsf{b}), (\mathsf{ac}, \mathsf{c})\}.$$

Otherwise, the simulation relation is

$$S_{2,1} = \{(\mathsf{ac}, \mathsf{a}), (\mathsf{bd}, \mathsf{b}), (\mathsf{ac}, \mathsf{c}), (\mathsf{bd}, \mathsf{d})\}.$$

All three are valid simulation relations, so the simulation relation is not unique.

14.4.5 Simulation vs. Language Containment

As with all abstraction-refinement relations, simulation is typically used to relate a simpler specification M_1 to a more complicated realization M_2. When M_1 simulates M_2, then the language of M_1 contains the language of M_2, but the guarantee is stronger than language containment. This fact is summarized in the following theorem.

Theorem 14.1. *Let M_1 simulate M_2. Then*

$$L(M_2) \subseteq L(M_1).$$

input: x: pure
output: y: $\{0, 1\}$

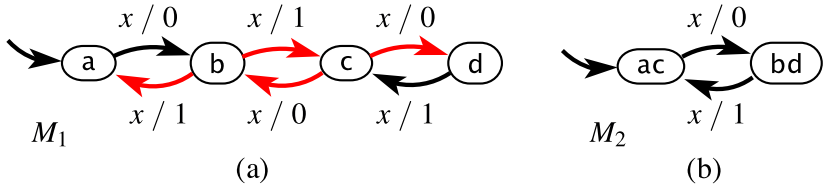


Figure 14.4: Two state machines that simulate each other, where there is more than one simulation relation.

Proof. This theorem is easy to prove. Consider a behavior $(x, y) \in L(M_2)$. We need to show that $(x, y) \in L(M_1)$.

Let the simulation relation be S. Find all possible execution traces for M_2

$$((x_0, s_0, y_0), (x_1, s_1, y_1), (x_2, s_2, y_2), \cdots),$$

that result in behavior (x, y). (If M_2 is deterministic, then there will be only one execution trace.) The simulation relation assures us that we can find an execution trace for M_1

$$((x_0, s'_0, y_0), (x_1, s'_1, y_1), (x_2, s'_2, y_2), \cdots),$$

where $(s_i, s'_i) \in S$, such that given input valuation x_i, M_1 produces y_i. Thus, $(x, y) \in L(M_1)$.

□

One use of this theorem is to show that M_1 does not simulate M_2 by showing that M_2 has behaviors that M_1 does not have.

Example 14.13: For the examples in Figure 14.2, M_2 does not simulate M_3. To see this, just note that the language of M_2 is a strict subset of the language of M_3,

$$L(M_2) \subset L(M_3),$$

meaning that M_3 has behaviors that M_2 does not have.

It is important to understand what the theorem says, and what it does not say. It does not say, for example, that if $L(M_2) \subseteq L(M_1)$ then M_1 simulates M_2. In fact, this statement is not true, as we have already shown with the examples in Figure 14.3. These two machines have the same language. The two machines are observably different despite the fact that their input/output behaviors are the same.

Of course, if M_1 and M_2 are deterministic and M_1 simulates M_2, then their languages are identical and M_2 simulates M_1. Thus, the simulation relation differs from language containment only for nondeterministic FSMs.

14.5 Bisimulation

It is possible to have two machines M_1 and M_2 where M_1 simulates M_2 and M_2 simulates M_1, and yet the machines are observably different. Note that by the theorem in the previous section, the languages of these two machines must be identical.

Example 14.14: Consider the two machines in Figure 14.5. These two machines simulate each other, with simulation relations as follows:

$$S_{2,1} = \{(\mathsf{e},\mathsf{a}), (\mathsf{f},\mathsf{b}), (\mathsf{h},\mathsf{b}), (\mathsf{j},\mathsf{b}), (\mathsf{g},\mathsf{c}), (\mathsf{i},\mathsf{d}), (\mathsf{k},\mathsf{c}), (\mathsf{m},\mathsf{d})\}$$

(M_1 simulates M_2), and

$$S_{1,2} = \{(\mathsf{a},\mathsf{e}), (\mathsf{b},\mathsf{j}), (\mathsf{c},\mathsf{k}), (\mathsf{d},\mathsf{m})\}$$

(M_2 simulates M_1). However, there is a situation in which the two machines will be observably different. In particular, suppose that the policies for making the

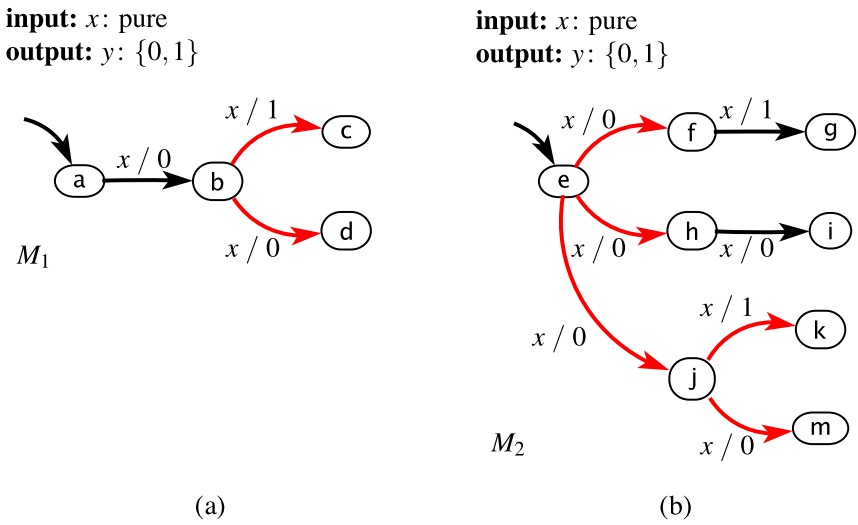


Figure 14.5: An example of two machines where M_1 simulates M_2, and M_2 simulates M_1, but they are not bisimilar.

nondeterministic choices for the two machines work as follows. In each reaction, they flip a coin to see which machine gets to move first. Given an input valuation, that machine makes a choice of move. The machine that moves second must be able to match all of its possible choices. In this case, the machines can end up in a state where one machine can no longer match all the possible moves of the other.

Specifically, suppose that in the first move M_2 gets to move first. It has three possible moves, and M_1 will have to match all three. Suppose it chooses to move to f or h. In the next round, if M_1 gets to move first, then M_2 can no longer match all of its possible moves.

Notice that this argument does not undermine the observation that these machines simulate each other. If in each round, M_2 always moves first, then M_1 will always be able to match its every move. Similarly, if in each round M_1 moves first, then M_2 can always match its every move (by always choosing to move to j in the first round). The observable difference arises from the ability to alternate which machines moves first.

To ensure that two machines are observably identical in all environments, we need a stronger equivalence relation called **bisimulation**. We say that M_1 is **bisimilar** to M_2 (or M_1 **bisimulates** M_2) if we can play the matching game modified so that in each round either machine can move first.

As in Section 14.4.2, we can use the formal model of nondeterministic FSMs to define a bisimulation relation. Let

$$\begin{aligned} M_1 &= (States_1, Inputs, Outputs, possibleUpdates_1, initialState_1), \text{ and} \\ M_2 &= (States_2, Inputs, Outputs, possibleUpdates_2, initialState_2). \end{aligned}$$

Assume the two machines are type equivalent. If either machine is deterministic, then its *possibleUpdates* function always returns a set with only one element in it. If M_1 bisimulates M_2, the simulation relation is given as a subset of $States_2 \times States_1$. The ordering here is not important because if M_1 bisimulates M_2, then M_2 bisimulates M_1.

We say that M_1 **bisimulates** M_2 if there is a subset $S \subseteq States_2 \times States_1$ such that

1. $(initialState_2, initialState_1) \in S$, and
2. If $(s_2, s_1) \in S$, then $\forall\, x \in Inputs$, and
   $\forall\, (s_2', y_2) \in possibleUpdates_2(s_2, x)$,
   there is a $(s_1', y_1) \in possibleUpdates_1(s_1, x)$ such that:
   (a) $(s_2', s_1') \in S$, and
   (b) $y_2 = y_1$, and
3. If $(s_2, s_1) \in S$, then $\forall\, x \in Inputs$, and
   $\forall\, (s_1', y_1) \in possibleUpdates_1(s_1, x)$,
   there is a $(s_2', y_2) \in possibleUpdates_2(s_2, x)$ such that:
   (a) $(s_2', s_1') \in S$, and
   (b) $y_2 = y_1$.

This set S, if it exists, is called the **bisimulation relation**. It establishes a correspondence between states in the two machines. If it does not exist, then M_1 does not bisimulate M_2.

14.6 Summary

In this chapter, we have considered three increasingly strong abstraction-refinement relations for FSMs. These relations enable designers to determine when one design can safely replace another, or when one design correctly implements a specification. The first relation is type refinement, which considers only the existence of input and output ports and their data types. The second relation is language refinement, which considers the sequences of valuations of inputs and outputs. The third relation is simulation, which considers the state trajectories of the machines. In all three cases, we have provided both a refinement relation and an equivalence relation. The strongest equivalence relation is bisimulation, which ensures that two nondeterministic FSMs are indistinguishable from each other.

Exercises

1. In Figure 14.6 are four pairs of actors. For each pair, determine whether
   - A and B are type equivalent,
   - A is a type refinement of B,
   - B is a type refinement of A, or
   - none of the above.

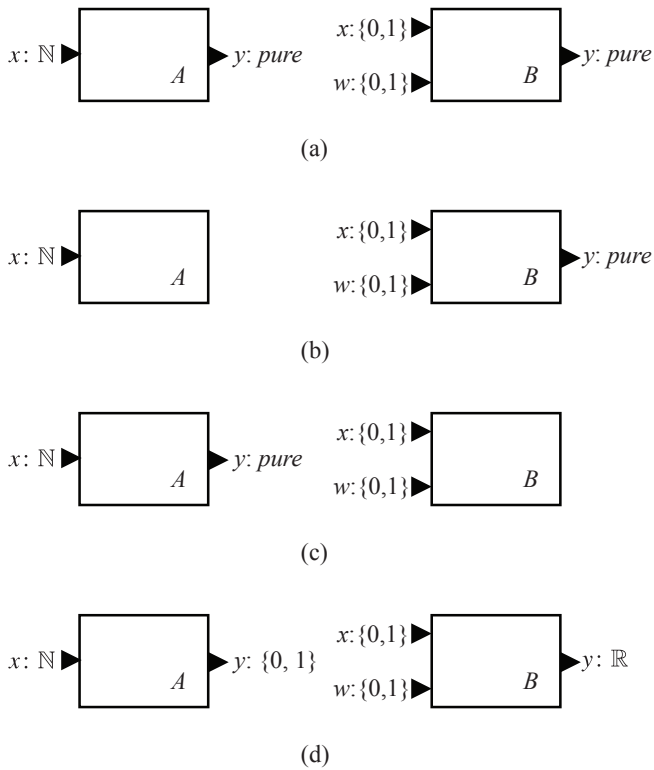


Figure 14.6: Four pairs of actors whose type refinement relationships are explored in Exercise 1.

2. In the box on page 384, a state machine M is given that accepts finite inputs x of the form (1), $(1,0,1)$, $(1,0,1,0,1)$, etc.

   (a) Write a regular expression that describes these inputs. You may ignore stuttering reactions.

   (b) Describe the output sequences in $L_a(M)$ in words, and give a regular expression for those output sequences. You may again ignore stuttering reactions.

   (c) Create a state machine that accepts *output* sequences of the form (1), $(1,0,1)$, $(1,0,1,0,1)$, etc. (see box on page 384). Assume the input x is pure and that whenever the input is present, a present output is produced. Give a deterministic solution if there is one, or explain why there is no deterministic solution. What *input* sequences does your machine accept?

3. The state machine in Figure 14.7 has the property that it outputs at least one 1 between any two 0's. Construct a two-state nondeterministic state machine that simulates this one and preserves that property. Give the simulation relation. Are the machines bisimilar?

4. Consider the FSM in Figure 14.8, which recognizes an input code. The state machine in Figure 14.9 also recognizes the same code, but has more states than the one in Figure 14.8. Show that it is equivalent by giving a bisimulation relation with the machine in Figure 14.8.

5. Consider the state machine in Figure 14.10. Find a bisimilar state machine with only two states, and give the bisimulation relation.

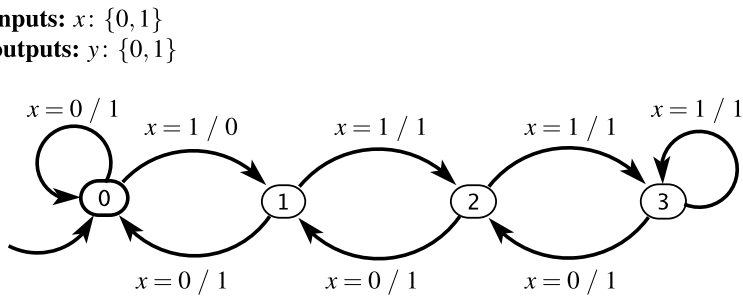


Figure 14.7: Machine that outputs at least one 1 between any two 0's.

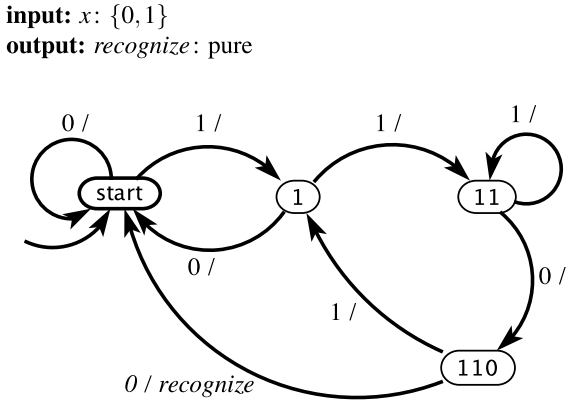


Figure 14.8: A machine that implements a code recognizer. It outputs *recognize* at the end of every input subsequence 1100; otherwise it outputs *absent*.

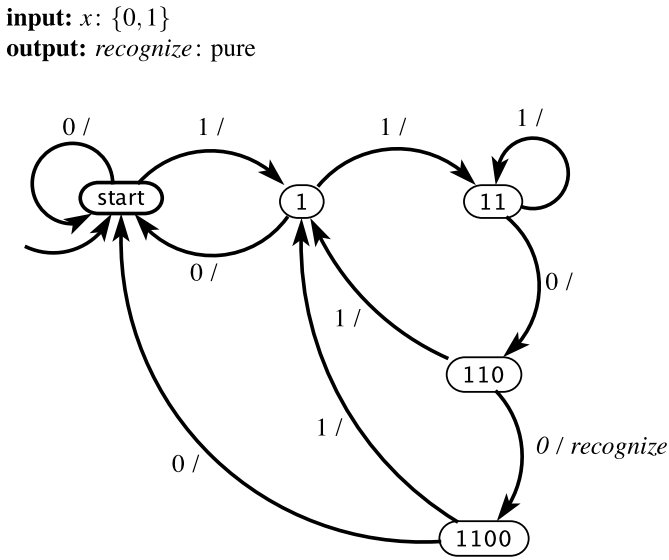


Figure 14.9: A machine that implements a recognizer for the same code as in Figure 14.8, but has more states.

6. You are told that state machine A has one input x, and one output y, both with type $\{1, 2\}$, and that it has states $\{a, b, c, d\}$. You are told nothing further. Do you have enough information to construct a state machine B that simulates A? If so, give such a state machine, and the simulation relation.

7. Consider a state machine with a pure input x, and output y of type $\{0, 1\}$. Assume the states are

$$States = \{a, b, c, d, e, f\},$$

and the initial state is a. The *update* function is given by the following table (ignoring stuttering):

(*currentState*, *input*)	(*nextState*, *output*)
(a, x)	$(b, 1)$
(b, x)	$(c, 0)$
(c, x)	$(d, 0)$
(d, x)	$(e, 1)$
(e, x)	$(f, 0)$
(f, x)	$(a, 0)$

   (a) Draw the state transition diagram for this machine.

   (b) Ignoring stuttering, give all possible behaviors for this machine.

   (c) Find a state machine with three states that is bisimilar to this one. Draw that state machine, and give the bisimulation relation.

8. For each of the following questions, give a short answer and justification.

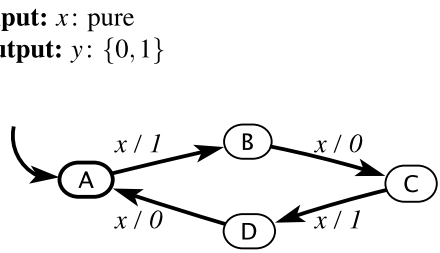


Figure 14.10: A machine that has more states than it needs.

(a) TRUE or FALSE: Consider a state machine A that has one input x, and one output y, both with type $\{1, 2\}$ and a single state s, with two self loops labeled $true/1$ and $true/2$. Then for any state machine B which has exactly the same inputs and outputs (along with types), A simulates B.

(b) TRUE or FALSE: Suppose that f is an arbitrary LTL formula that holds for state machine A, and that A simulates another state machine B. Then we can safely assert that f holds for B.

(c) TRUE or FALSE: Suppose that A are B are two type-equivalent state machines, and that f is an LTL formula where the atomic propositions refer only to the inputs and outputs of A and B, not to their states. If the LTL formula f holds for state machine A, and A simulates state machine B, then f holds for B.

15 Reachability Analysis and Model Checking

15.1 Open and Closed Systems 405
15.2 Reachability Analysis 406
15.2.1 Verifying $\mathbf{G}p$ 407
15.2.2 Explicit-State Model Checking 409
15.2.3 Symbolic Model Checking 411
15.3 Abstraction in Model Checking 413
15.4 Model Checking Liveness Properties 417
15.4.1 Properties as Automata 418
15.4.2 Finding Acceptance Cycles 420
15.5 Summary 423
Sidebar: Probing Further: Model Checking in Practice 424
Exercises 425

Chapters 13 and 14 have introduced techniques for formally specifying properties and models of systems, and for comparing such models. In this chapter, we will study algorithmic techniques for **formal verification** — the problem of checking whether a system satisfies its formal specification in its specified operating environment. In particular, we study a technique called **model checking**. Model checking is an algorithmic method for determining whether a system satisfies a formal specification expressed as a temporal

logic formula. It was introduced by Clarke and Emerson (1981) and Queille and Sifakis (1981), which earned the creators the 2007 ACM Turing Award, the highest honor in Computer Science.

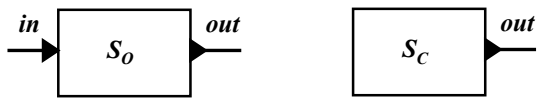
Central to model checking is the notion of the set of reachable states of a system. **Reachability analysis** is the process of computing the set of reachable states of a system. This chapter presents basic algorithms and ideas in reachability analysis and model checking. These algorithms are illustrated using examples drawn from embedded systems design, including verification of high-level models, sequential and concurrent software, as well as control and robot path planning. Model checking is a large and active area of research, and a detailed treatment of the subject is out of the scope of this chapter; we refer the interested reader to Clarke et al. (1999) and Holzmann (2004) for an in-depth introduction to this field.

15.1 Open and Closed Systems

A **closed system** is one with no inputs. An **open system**, in contrast, is one that maintains an ongoing interaction with its environment by receiving inputs and (possibly) generating output to the environment. Figure 15.1 illustrates these concepts.

Techniques for formal verification are typically applied to a model of the closed system M obtained by composing the model of the system S that is to be verified with a model of its environment E. S and E are typically open systems, where all inputs to S are generated by E and vice-versa. Thus, as shown in Figure 15.2, there are three inputs to the verification process:

- A model of the system to be verified, S;
- A model of the environment, E, and



(a) Open system (b) Closed system

Figure 15.1: Open and closed systems.

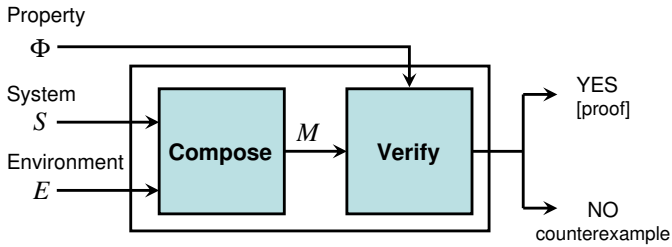


Figure 15.2: Formal verification procedure.

- The property to be verified Φ.

The verifier generates as output a YES/NO answer, indicating whether or not S satisfies the property Φ in environment E. Typically, a NO output is accompanied by a counterexample, also called an **error trace**, which is a trace of the system that indicates how Φ is violated. Counterexamples are very useful aids in the debugging process. Some formal verification tools also include a proof or certificate of correctness with a YES answer; such an output can be useful for **certification** of system correctness.

The form of composition used to combine system model S with environment model E depends on the form of the interaction between system and environment. Chapters 5 and 6 describe several ways to compose state machine models. All of these forms of composition can be used in generating a verification model M from S and E. Note that M can be nondeterministic.

For simplicity, in this chapter we will assume that system composition has already been performed using one of the techniques presented in Chapters 5 and 6. All algorithms discussed in the following sections will operate on the combined verification model M, and will be concerned with answering the question of whether M satisfies property Φ. Additionally, we will assume that Φ is specified as a property in linear temporal logic.

15.2 Reachability Analysis

We consider first a special case of the model checking problem which is useful in practice. Specifically, we assume that M is a finite-state machine and Φ is an LTL formula of the form $\mathbf{G}p$, where p is a proposition. Recall from Chapter 13 that $\mathbf{G}p$ is the temporal logic

formula that holds in a trace when the proposition p holds in every state of that trace. As we have seen in Chapter 13, several system properties are expressible as $\mathbf{G}p$ properties.

We will begin in Section 15.2.1 by illustrating how computing the reachable states of a system enables one to verify a $\mathbf{G}p$ property. In Section 15.2.2 we will describe a technique for reachability analysis of finite-state machines based on explicit enumeration of states. Finally, in Section 15.2.3, we will describe an alternative approach to analyze systems with very large state spaces.

15.2.1 Verifying $\mathrm{G}p$

In order for a system M to satisfy $\mathbf{G}p$, where p is a proposition, every trace exhibitable by M must satisfy $\mathbf{G}p$. This property can be verified by enumerating all states of M and checking that every state satisfies p.

When M is finite-state, in theory, such enumeration is always possible. As shown in Chapter 3, the state space of M can be viewed as a directed graph where the nodes of the graph correspond to states of M and the edges correspond to transitions of M. This graph is called the **state graph** of M, and the set of all states is called its state space. With this graph-theoretic viewpoint, one can see that checking $\mathbf{G}p$ for a finite-state system M corresponds to traversing the state graph for M, starting from the initial state and checking that every state reached in this traversal satisfies p. Since M has a finite number of states, this traversal must terminate.

Example 15.1: Let the system S be the traffic light controller of Figure 3.10 and its environment E be the pedestrian model shown in Figure 3.11. Let M be the synchronous composition of S and E as shown in Figure 15.3. Observe that M is a closed system. Suppose that we wish to verify that M satisfies the property

$$\mathbf{G}\ \neg(\mathsf{green} \wedge \mathsf{crossing})$$

In other words, we want to verify that it is never the case that the traffic light is green while pedestrians are crossing.

The composed system M is shown in Figure 15.4 as an extended FSM. Note that M has no inputs or outputs. M is finite-state, with a total of 188 states (using a similar calculation to that in Example 3.12). The graph in Figure 15.4 is not

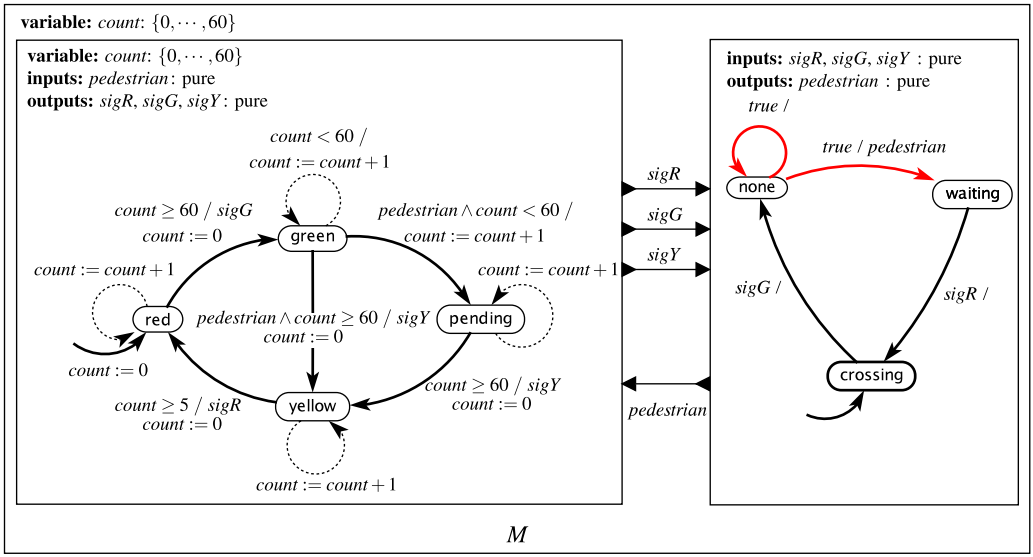


Figure 15.3: Composition of traffic light controller (Figure 3.10) and pedestrian model (Figure 3.11).

the full state graph of M, because each node represents a set of states, one for each different value of *count* in that node. However, through visual inspection of this graph we can check for ourselves that no state satisfies the proposition (green ∧ crossing), and hence every trace satisfies the LTL property $\mathbf{G}\,\neg$(green ∧ crossing).

In practice, the seemingly simple task of verifying whether a finite-state system M satisfies a $\mathbf{G}p$ property is not as straightforward as in the previous example for the following reasons:

- Typically, one starts only with the initial state and transition function, and the state graph must be constructed on the fly.

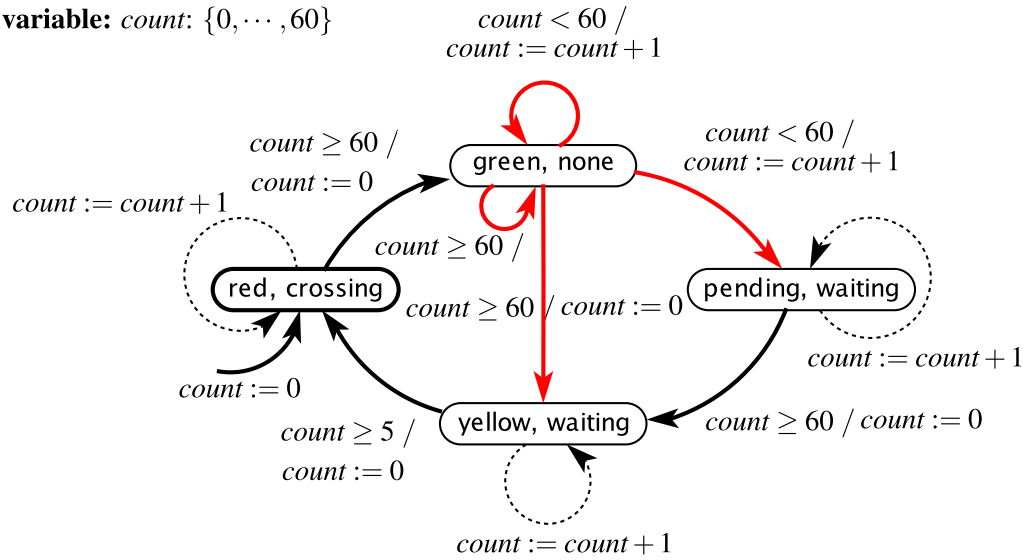


Figure 15.4: Extended state machine obtained from synchronous-reactive composition of traffic light controller and pedestrian models. Note that this is nondeterministic.

- The system might have a huge number of states, possibly exponential in the size of the syntactic description of M. As a consequence, the state graph cannot be represented using traditional data structures such as an adjacency or incidence matrix.

The next two sections describe how these challenges can be handled.

15.2.2 Explicit-State Model Checking

In this section, we discuss how to compute the reachable state set by generating and traversing the state graph on the fly.

First, recall that the system of interest M is closed, finite-state, and can be nondeterministic. Since M has no inputs, its set of possible next states is a function of its current state alone. We denote this transition relation of M by δ, which is only a function of the current state of M, in contrast to the *possibleUpdates* function introduced in Chapter 3 which is

also a function of the current input. Thus, $\delta(s)$ is the set of possible next states from state s of M.

Algorithm 15.1 computes the set of reachable states of M, given its initial state s_0 and transition relation δ. Procedure **DFS_Search** performs a depth-first traversal of the state graph of M, starting with state s_0. The graph is generated on-the-fly by repeatedly applying δ to states visited during the traversal.

Input : Initial state s_0 and transition relation δ for closed finite-state system M
Output: Set R of reachable states of M

```
1  Initialize: Stack Σ to contain a single state s0; Current set of reached
   states R := {s0}.
2  DFS_Search() {
3  while Stack Σ is not empty do
4      Pop the state s at the top of Σ
5      Compute δ(s), the set of all states reachable from s in one
       transition
6      for each s' ∈ δ(s) do
7          if s' ∉ R then
8              R := R ∪ {s'}
9              Push s' onto Σ
10         end
11     end
12 end
13 }
```

Algorithm 15.1: Computing the reachable state set by depth-first explicit-state search.

The main data structures required by the algorithm are Σ, the stack storing the current path in the state graph being explored from s_0, and R, the current set of states reached during traversal. Since M is finite-state, at some point all states reachable from s_0 will be in R, which implies that no new states will be pushed onto Σ and thus Σ will become empty. Hence, procedure **DFS_Search** terminates and the value of R at the end of the procedure is the set of all reachable states of M.

The space and time requirements for this algorithm are linear in the size of the state graph (see Appendix B for an introduction to such complexity notions). However, the number of nodes and edges in the state graph of M can be exponential in the size of the descriptions of S and E. For example, if S and E together have 100 Boolean state variables (a small

number in practice!), the state graph of M can have a total of 2^{100} states, far more than what contemporary computers can store in main memory. Therefore, explicit-state search algorithms such as **DFS_Search** must be augmented with **state compression** techniques. Some of these techniques are reviewed in the sidebar on page 424.

A challenge for model checking concurrent systems is the **state-explosion problem**. Recall that the state space of a composition of k finite-state systems $M_1, M_2, \ldots, M_k$ (say, using synchronous composition), is the cartesian product of the state spaces of $M_1, M_2, \ldots, M_k$. In other words, if $M_1, M_2, \ldots, M_k$ have $n_1, n_2, \ldots, n_k$ states respectively, their composition can have $\Pi_{i=1}^{k} n_i$ states. It is easy to see that the number of states of a concurrent composition of k components grows exponentially with k. Explicitly representing the state space of the composite system does not scale. In the next section, we will introduce techniques that can mitigate this problem in some cases.

15.2.3 Symbolic Model Checking

The key idea in **symbolic model checking** is to represent a set of states *symbolically* as a propositional logic formula, rather than explicitly as a collection of individual states. Specialized data structures are often used to efficiently represent and manipulate such formulas. Thus, in contrast to explicit-state model checking, in which individual states are manipulated, symbolic model checking operates on sets of states.

Algorithm 15.2 (**Symbolic_Search**) is a symbolic algorithm for computing the set of reachable states of a closed, finite-state system M. This algorithm has the same input-output specification as the previous explicit-state algorithm **DFS_Search**; however, all operations in **Symbolic_Search** are set operations.

In algorithm **Symbolic_Search**, R represents the entire set of states reached at any point in the search, and R_{new} represents the *new* states generated at that point. When no more new states are generated, the algorithm terminates, with R storing all states reachable from s_0. The key step of the algorithm is line 5, in which R_{new} is computed as the set of all states s' reachable from any state s in R in one step of the transition relation δ. This operation is called **image computation**, since it involves computing the image of the function δ. Efficient implementations of image computation that directly operate on propositional logic formulas are central to symbolic reachability algorithms. Apart from image computation, the key set operations in **Symbolic_Search** include set union and emptiness checking.

Input : Initial state s_0 and transition relation δ for closed finite-state system M, represented symbolically
Output: Set R of reachable states of M, represented symbolically

1 **Initialize:** *Current set of reached states* $R = \{s_0\}$
2 **Symbolic_Search**() {
3 $R_{\text{new}} = R$
4 **while** $R_{\text{new}} \neq \emptyset$ **do**
5 $\quad R_{\text{new}} := \{s' \mid \exists s \in R \text{ s.t. } s' \in \delta(s) \wedge s' \notin R\}$
6 $\quad R := R \cup R_{\text{new}}$
7 **end**
8 }

Algorithm 15.2: Computing the reachable state set by symbolic search.

Example 15.2: We illustrate symbolic reachability analysis using the finite-state system in Figure 15.4.

To begin with, we need to introduce some notation. Let v_l be a variable denoting the state of the traffic light controller FSM S at the start of each reaction; i.e., $v_l \in \{\mathsf{green}, \mathsf{yellow}, \mathsf{red}, \mathsf{pending}\}$. Similarly, let v_p denote the state of the pedestrian FSM E, where $v_p \in \{\mathsf{crossing}, \mathsf{none}, \mathsf{waiting}\}$.

Given this notation, the initial state set $\{s_0\}$ of the composite system M is represented as the following propositional logical formula:

$$v_l = \mathsf{red} \wedge v_p = \mathsf{crossing} \wedge \mathit{count} = 0$$

From s_0, the only enabled outgoing transition is the self-loop on the initial state of the extended FSM in Figure 15.4. Thus, after one step of reachability computation, the set of reached states R is represented by the following formula:

$$v_l = \mathsf{red} \wedge v_p = \mathsf{crossing} \wedge 0 \leq \mathit{count} \leq 1$$

After two steps, R is given by

$$v_l = \mathsf{red} \wedge v_p = \mathsf{crossing} \wedge 0 \leq \mathit{count} \leq 2$$

and after k steps, $k \leq 60$, R is represented by the formula

$$v_l = \mathsf{red} \wedge v_p = \mathsf{crossing} \wedge 0 \leq \mathit{count} \leq k$$

On the 61st step, we exit the state $(\mathsf{red}, \mathsf{crossing})$, and compute R as

$$\begin{aligned} & v_l = \mathsf{red} \wedge v_p = \mathsf{crossing} \wedge 0 \leq \mathit{count} \leq 60 \\ \vee\ & v_l = \mathsf{green} \wedge v_p = \mathsf{none} \wedge \mathit{count} = 0 \end{aligned}$$

Proceeding similarly, the set of reachable states R is grown until there is no further change. The final reachable set is represented as:

$$\begin{aligned} & v_l = \mathsf{red} \wedge v_p = \mathsf{crossing} \wedge 0 \leq \mathit{count} \leq 60 \\ \vee\ & v_l = \mathsf{green} \wedge v_p = \mathsf{none} \wedge 0 \leq \mathit{count} \leq 60 \\ \vee\ & v_l = \mathsf{pending} \wedge v_p = \mathsf{waiting} \wedge 0 < \mathit{count} \leq 60 \\ \vee\ & v_l = \mathsf{yellow} \wedge v_p = \mathsf{waiting} \wedge 0 \leq \mathit{count} \leq 5 \end{aligned}$$

In practice, the symbolic representation is much more compact than the explicit one. The previous example illustrates this nicely because a large number of states are compactly represented by inequalities like $0 < \mathit{count} \leq 60$. Computer programs can be designed to operate directly on the symbolic representation. Some examples of such programs are given in the box on page 424.

Symbolic model checking has been used successfully to address the state-explosion problem for many classes of systems, most notably for hardware models. However, in the worst case, even symbolic set representations can be exponential in the number of system variables.

15.3 Abstraction in Model Checking

A challenge in model checking is to work with the simplest abstraction of a system that will provide the required proofs of safety. Simpler abstractions have smaller state spaces and can be checked more efficiently. The challenge, of course, is to know what details to omit from the abstraction.

The part of the system to be abstracted away depends on the property to be verified. The following example illustrates this point.

Example 15.3: Consider the traffic light system M in Figure 15.4. Suppose that, as in Example 15.1 we wish to verify that M satisfies the property

$$\mathbf{G}\ \neg(\mathsf{green} \wedge \mathsf{crossing})$$

Suppose we abstract the variable *count* away from M by hiding all references to *count* from the model, including all guards mentioning it and all updates to it. This generates the abstract model M_{abs} shown in Figure 15.5.

We observe that this abstract M_{abs} exhibits more behaviors than M. For instance, from the state $(\mathsf{yellow}, \mathsf{waiting})$ we can take the self-loop transition forever, staying in that state perennially, even though in the actual system M this state must be exited within five clock ticks. Moreover, every behavior of M can be exhibited by M_{abs}.

The interesting point is that, even with this approximation, we can prove that M_{abs} satisfies $\mathbf{G}\ \neg(\mathsf{green} \wedge \mathsf{crossing})$. The value of *count* is irrelevant for this property.

Notice that while M has 188 states, M_{abs} has only 4 states. Reachability analysis on M_{abs} is far easier than for M as we have far fewer states to explore.

There are several ways to compute an abstraction. One of the simple and extremely useful approaches is called **localization reduction** or **localization abstraction** (Kurshan (1994)). In localization reduction, parts of the design model that are irrelevant to the property being checked are abstracted away by hiding a subset of state variables. Hiding a variable corresponds to freeing that variable to evolve arbitrarily. It is the form of abstraction used in Example 15.3 above, where *count* is allowed to change arbitrarily, and all transitions are made independent of the value of *count*.

Example 15.4: Consider the multithreaded program given below (adapted from Ball et al. (2001)). The procedure `lock_unlock` executes a loop within which it acquires a lock, then calls the function `randomCall`, based on whose result it either releases the lock and executes another loop iteration, or it quits the loop (and then releases the lock). The execution of another loop iteration is

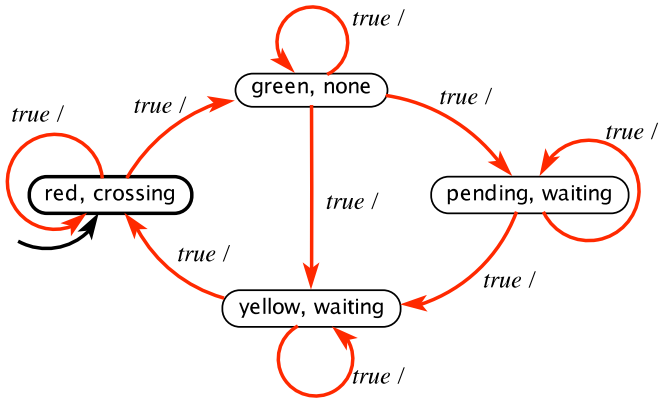


Figure 15.5: Abstraction of the traffic light system in Figure 15.4.

ensured by incrementing `new`, so that the condition `old != new` evaluates to true.

```
1  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2  unsigned int old, new;
3
4  void lock_unlock() {
5    do {
6      pthread_mutex_lock(&lock);
7      old = new;
8      if (randomCall()) {
9        pthread_mutex_unlock(&lock);
10       new++;
11     }
12   } while (old != new)
13   pthread_mutex_unlock(&lock);
14 }
```

Suppose the property we want to verify is that the code does not attempt to call `pthread_mutex_lock` twice in a row. Recall from Section 11.2.4 how the system can deadlock if a thread becomes permanently blocked trying to acquire a lock. This could happen in the above example if the thread, already holding lock `lock`, attempts to acquire it again.

If we model this program exactly, without any abstraction, then we need to reason about all possible values of `old` and `new`, in addition to the remaining state of the program. Assuming a word size of 32 in this system, the size of the state space is roughly $2^{32} \times 2^{32} \times n$, where 2^{32} is the number of values of `old` and `new`, and n denotes the size of the remainder of the state space.

However, it is not necessary to reason about the precise values of `old` and `new` to prove that this program is correct. Assume, for this example, that our programming language is equipped with a `boolean` type. Assume further that the program can perform nondeterministic assignments. Then, we can generate the following abstraction of the original program, written in C-like syntax, where the Boolean variable `b` represents the predicate `old == new`.

```
1  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2  boolean b; // b represents the predicate (old == new)
3  void lock_unlock() {
4    do {
5        pthread_mutex_lock(&lock);
6        b = true;
7        if (randomCall()) {
8          pthread_mutex_unlock(&lock);
9          b = false;
10       }
11   } while (!b)
12   pthread_mutex_unlock(&lock);
13 }
```

It is easy to see that this abstraction retains just enough information to show that the program satisfies the desired property. Specifically, the lock will not be acquired twice because the loop is only iterated if `b` is set to false, which implies that the lock was released before the next attempt to acquire.

Moreover, observe that size of the state space to be explored has reduced to simply $2n$. This is the power of using the "right" abstraction.

A major challenge for formal verification is to *automatically* compute simple abstractions. An effective and widely-used technique is **counterexample-guided abstraction refinement** (**CEGAR**), first introduced by Clarke et al. (2000). The basic idea (when using localization reduction) is to start by hiding almost all state variables except those referenced by the temporal logic property. The resulting abstract system will have more

behaviors than the original system. Therefore, if this abstract system satisfies an LTL formula Φ (i.e., each of its behaviors satisfies Φ), then so does the original. However, if the abstract system does not satisfy Φ, the model checker generates a counterexample. If this counterexample is a counterexample for the original system, the process terminates, having found a genuine counterexample. Otherwise, the CEGAR approach analyzes this counterexample to infer which hidden variables must be made visible, and with these additional variables, recomputes an abstraction. The process continues, terminating either with some abstract system being proven correct, or generating a valid counterexample for the original system.

The CEGAR approach and several follow-up ideas have been instrumental in driving progress in the area of software model checking. We review some of the key ideas in the sidebar on page 424.

15.4 Model Checking Liveness Properties

So far, we have restricted ourselves to verifying properties of the form $\mathbf{G}p$, where p is an atomic proposition. An assertion that $\mathbf{G}p$ holds for all traces is a very restricted kind of safety property. However, as we have seen in Chapter 13, several useful system properties are not safety properties. For instance, the property stating that "the robot must visit location A" is a liveness property: if visiting location A is represented by proposition q, then this property is an assertion that $\mathbf{F}q$ must hold for all traces. In fact, several problems, including path planning problems for robotics and progress properties of distributed and concurrent systems can be stated as liveness properties. It is therefore useful to extend model checking to handle this class of properties.

Properties of the form $\mathbf{F}p$, though liveness properties, can be partially checked using the techniques introduced earlier in this chapter. Recall from Chapter 13 that $\mathbf{F}p$ holds for a trace if and only if $\neg\mathbf{G}\neg p$ holds for the same trace. In words, "p is true some time in the future" iff "$\neg p$ is always false." Therefore, we can attempt to verify that the system satisfies $\mathbf{G}\neg p$. If the verifier asserts that $\mathbf{G}\neg p$ holds for all traces, then we know that $\mathbf{F}p$ does not hold for any trace. On the other hand, if the verifier outputs "NO", then the accompanying counterexample provides a witness exhibiting how p may become true eventually. This witness provides one trace for which $\mathbf{F}p$ holds, but it does not prove that $\mathbf{F}p$ holds for all traces (unless the machine is deterministic).

More complete checks and more complicated liveness properties require a more sophisticated approach. Briefly, one approach used in explicit-state model checking of LTL properties is as follows:

1. Represent the negation of the property Φ as an automaton B, where certain states are labeled as accepting states.
2. Construct the synchronous composition of the property automaton B and the system automaton M. The accepting states of the property automaton induce accepting states of the product automaton M_B.
3. If the product automaton M_B can visit an accepting state infinitely often, then it indicates that M does not satisfy Φ; otherwise, M satisfies Φ.

The above approach is known as the **automata-theoretic approach to verification**. We give a brief introduction to this subject in the rest of this section. Further details may be found in the seminal papers on this topic (Wolper et al. (1983); Vardi and Wolper (1986)) and the book on the SPIN model checker (Holzmann (2004)).

15.4.1 Properties as Automata

Consider the first step of viewing properties as automata. Recall the material on omega-regular languages introduced in the box on page 386. The theory of Büchi automata and omega-regular languages, briefly introduced there, is relevant for model checking liveness properties. Roughly speaking, an LTL property Φ has a one-to-one correspondence with a set of behaviors that satisfy Φ. This set of behaviors constitutes the language of the Büchi automaton corresponding to Φ.

For the LTL model checking approach we describe here, if Φ is the property that the system must satisfy, then we represent its negation $\neg\Phi$ as a Büchi automaton. We present some illustrative examples below.

Example 15.5: Suppose that an FSM M_1 models a system that executes forever and produces a pure output h (for heartbeat), and that it is required to produce this output at least once every three reactions. That is, if in two successive reactions it fails to produce the output h, then in the third it must.

We can formulate this property in LTL as the property Φ_1 below:

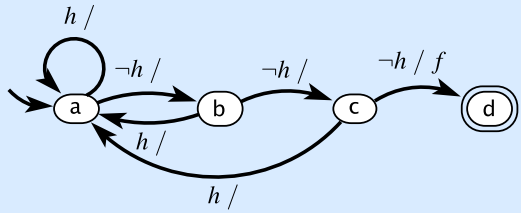
$$\mathbf{G}(h \vee \mathbf{X}h \vee \mathbf{X}^2 h)$$

and the negation of this property is

$$\mathbf{F}(\neg h \wedge \mathbf{X}\neg h \wedge \mathbf{X}^2 \neg h)$$

The Büchi automaton B_1 corresponding to the negation of the desired property is given below:

input: h: pure
output: f: pure



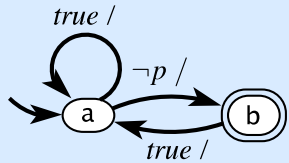
Let us examine this automaton. The language accepted by this automaton includes all behaviors that enter and stay in state d. Equivalently, the language includes all behaviors that produce a *present* output on f in some reaction. When we compose the above machine with M_1, if the resulting composite machine can never produce $f = present$, then the language accepted by the composite machine is empty. If we can prove that the language is empty, then we have proved that M produces the heartbeat h at least once every three reactions.

Observe that the property Φ_1 in the above example is in fact a safety property. We give an example of a liveness property below.

Example 15.6: Suppose that the FSM M_2 models a controller for a robot that must locate a room and stay there forever. Let p be the proposition that becomes true when the robot is in the target room. Then, the desired property Φ_2 can be expressed in LTL as $\mathbf{FG}p$.

The negation of this property is $\mathbf{GF}\neg p$. The Büchi automaton B_2 corresponding to this negated property is given below:

input: p: pure



Notice that all accepting behaviors of B_2 correspond to those where $\neg p$ holds infinitely often. These behaviors correspond to a cycle in the state graph for the product automaton where state b of B_2 is visited repeatedly. This cycle is known as an **acceptance cycle**.

Liveness properties of the form $\mathbf{G}\,\mathbf{F}p$ also occur naturally as specifications. This form of property is useful in stating **fairness** properties which assert that certain desirable properties hold infinitely many times, as illustrated in the following example.

Example 15.7: Consider a traffic light system such as that in Example 3.10. We may wish to assert that the traffic light becomes green infinitely many times in any execution. In other words, the state green is visited infinitely often, which can be expressed as $\Phi_3 = \mathbf{GF}$ green.

The automaton corresponding to Φ_3 is identical to that for the negation of Φ_2 in Example 15.6 above with $\neg p$ replaced by green. However, in this case the accepting behaviors of this automaton are the desired behaviors.

Thus, from these examples we see that the problem of detecting whether a certain accepting state s in an FSM can be visited infinitely often is the workhorse of explicit-state model checking of LTL properties. We next present an algorithm for this problem.

15.4.2 Finding Acceptance Cycles

We consider the following problem:

> Given a finite-state system M, can an accepting state s_a of M be visited infinitely often?

Put another way, we seek an algorithm to check whether (i) state s_a is reachable from the initial state s_0 of M, and (ii) s_a is reachable from itself. Note that asking whether a state *can* be visited infinitely often is not the same as asking whether it *must* be visited infinitely often.

The graph-theoretic viewpoint is useful for this problem, as it was in the case of $\mathbf{G}p$ discussed in Section 15.2.1. Assume for the sake of argument that we have the entire state graph constructed a priori. Then, the problem of checking whether state s_a is reachable from s_0 is simply a graph traversal problem, solvable for example by depth-first search (DFS). Further, the problem of detecting whether s_a is reachable from itself amounts to checking whether there is a cycle in the state graph containing that state.

The main challenges for solving this problem are similar to those discussed in Section 15.2.1: we must perform this search on-the-fly, and we must deal with large state spaces.

The **nested depth-first search** (nested DFS) algorithm, which is implemented in the SPIN model checker (Holzmann (2004)), solves this problem and is shown as Algorithm 15.3. The algorithm begins by calling the procedure called Nested_DFS_Search with argument 1, as shown in the Main function at the bottom. M_B is obtained by composing the original closed system M with the automaton B representing the negation of LTL formula Φ.

As the name suggests, the idea is to perform two depth-first searches, one nested inside the other. The first DFS identifies a path from initial state s_0 to the target accepting state s_a. Then, from s_a we start another DFS to see if we can reach s_a again. The variable `mode` is either 1 or 2 depending on whether we are performing the first DFS or the second. Stacks Σ_1 and Σ_2 are used in the searches performed in modes 1 and 2 respectively. If s_a is encountered in the second DFS, the algorithm generates as output the path leading from s_0 to s_a with a loop on s_a. The path from s_0 to s_a is obtained simply by reading off the contents of stack Σ_1. Likewise, the cycle from s_a to itself is obtained from stack Σ_2. Otherwise, the algorithm reports failure.

Search optimization and state compression techniques that are used in explicit-state reachability analysis can be used with nested DFS also. Further details are available in Holzmann (2004).

Input : Initial state s_0 and transition relation δ for automaton M_B; Target accepting state s_a of M_B
Output: Acceptance cycle containing s_a, if one exists

```
1  Initialize: (i) Stack Σ1 to contain a single state s0, and stack Σ2 to be empty;
   (ii) Two sets of reached states R1 := R2 := {s0}; (iii) Flag found := false.
2  Nested_DFS_Search(Mode mode) {
3  while Stack Σ_mode is not empty do
4      Pop the state s at the top of Σ_mode
5      if (s = s_a and mode = 1) then
6          Push s onto Σ2
7          Nested_DFS_Search(2)
8          return
9      end
10     Compute δ(s), the set of all states reachable from s in one transition
11     for each s' ∈ δ(s) do
12         if (s' = s_a and mode = 2) then
13             Output path to s_a with acceptance cycle using contents of
               stacks Σ1 and Σ2
14             found := true
15             return
16         end
17         if s' ∉ R_mode then
18             R_mode := R_mode ∪ {s'}
19             Push s' onto Σ_mode
20             Nested_DFS_Search(mode)
21         end
22     end
23 end
24 }
25 Main() {
26     Nested_DFS_Search(1)
27     if (found = false) then Output "no acceptance cycle with s_a" end }
28
```

Algorithm 15.3: Nested depth-first search algorithm.

15.5 Summary

This chapter gives some basic algorithms for formal verification, including model checking, a technique for verifying if a finite-state system satisfies a property specified in temporal logic. Verification operates on closed systems, which are obtained by composing a system with its operating environment. The first key concept is that of reachability analysis, which verifies properties of the form $\mathbf{G}p$. The concept of abstraction, central to the scalability of model checking, is also discussed in this chapter. This chapter also shows how explicit-state model checking algorithms can handle liveness properties, where a crucial concept is the correspondence between properties and automata.

Probing Further: Model Checking in Practice

Several tools are available for computing the set of reachable states of a finite-state system and checking that they satisfy specifications in temporal logic. One such tool is **SMV** (symbolic model verifier), which was first developed at Carnegie Mellon University by Kenneth McMillan. SMV was the first model checking tool to use binary decision diagrams (**BDD**s), a compact data structure introduced by Bryant (1986) for representing a Boolean function. The use of BDDs has proved instrumental in enabling analysis of more complex systems. Current symbolic model checkers also rely heavily on Boolean satisfiability (SAT) solvers (see Malik and Zhang (2009)), which are programs for deciding whether a propositional logic formula can evaluate to true. One of the first uses of SAT solvers in model checking was for **bounded model checking** (see Biere et al. (1999)), where the transition relation of the system is unrolled only a bounded number of times. A few different versions of SMV are available online (see for example `http://nusmv.fbk.eu/`).

The SPIN model checker (Holzmann, 2004) developed in the 1980's and 1990's at Bell Labs by Gerard Holzmann and others, is another leading tool for model checking (see `http://www.spinroot.com/`). Rather than directly representing models as communicating FSMs, it uses a specification language (called Promela, for process meta language) that enables specifications that closely resemble multithreaded programs. SPIN incorporates state-compression techniques such as **hash compaction** (the use of hashing to reduce the size of the stored state set) and **partial-order reduction** (a technique to reduce the number of reachable states to be explored by considering only a subset of the possible process interleavings).

Automatic abstraction has played a big role in applying model checking directly to software. An example of abstraction-based software model checking is the **SLAM** system developed at Microsoft Research (Ball and Rajamani, 2001; Ball et al., 2011). SLAM combines CEGAR with a particular form of abstraction called predicate abstraction, in which predicates in a program are abstracted to Boolean variables. A key step in these techniques is checking whether a counterexample generated on the abstract model is in fact a true counterexample. This check is performed using satisfiability solvers for logics richer than propositional logic. These solvers are called **SAT-based decision procedures** or **satisfiability modulo theories** (**SMT**) solvers (for more details, see Barrett et al. (2009)).

More recently, techniques based on **inductive learning**, that is, generalization from sample data, have started playing an important role in formal verification (see Seshia (2015) for an exposition of this topic).

Exercises

1. Consider the system M modeled by the hierarchical state machine of Figure 13.2, which models an interrupt-driven program.

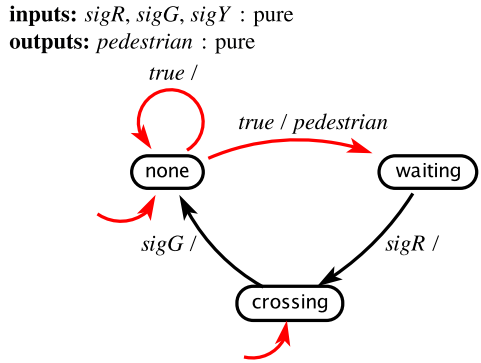
   Model M in the modeling language of a verification tool (such as SPIN). You will have to construct an environment model that asserts the interrupt. Use the verification tool to check whether M satisfies ϕ, the property stated in Exercise 5:

   ϕ: The main program eventually reaches program location C.

   Explain the output you obtain from the verification tool.

2. Figure 15.3 shows the synchronous-reactive composition of the traffic light controller of Figure 3.10 and the pedestrian model of Figure 3.11.

   Consider replacing the pedestrian model in Figure 15.3 with the alternative model given below where the initial state is nondeterministically chosen to be one of none or crossing:



   (a) Model the composite system in the modeling language of a verification tool (such as SPIN). How many reachable states does the combined system have? How many of these are initial states?

   (b) Formulate an LTL property stating that every time a pedestrian arrives, eventually the pedestrian is allowed to cross (i.e., the traffic light enters state red).

   (c) Use the verification tool to check whether the model constructed in part (a) satisfies the LTL property specified in part (b). Explain the output of the tool.

3. The notion of reachability has a nice symmetry. Instead of describing all states that are reachable from some initial state, it is just as easy to describe all states from which some state can be reached. Given a finite-state system M, the **backward reachable states** of a set F of states is the set B of all states from which some state in F can be reached. The following algorithm computes the set of backward reachable states for a given set of states F:

Input : A set F of states and transition relation δ for closed finite-state system M
Output: Set B of backward reachable states from F in M

1 **Initialize:** $B := F$
2 $B_{\text{new}} := B$
3 **while** $B_{\text{new}} \neq \emptyset$ **do**
4 $\quad B_{\text{new}} := \{s \mid \exists s' \in B \text{ s.t. } s' \in \delta(s) \wedge s \notin B\}$
5 $\quad B := B \cup B_{\text{new}}$
6 **end**

Explain how this algorithm can check the property $\mathbf{G}p$ on M, where p is some property that is easily checked for each state s in M. You may assume that M has exactly one initial state s_0.

16 Quantitative Analysis

16.1 Problems of Interest 428
16.1.1 Extreme-Case Analysis 429
16.1.2 Threshold Analysis 429
16.1.3 Average-Case Analysis 430
16.2 Programs as Graphs 430
16.2.1 Basic Blocks 431
16.2.2 Control-Flow Graphs 432
16.2.3 Function Calls 432
16.3 Factors Determining Execution Time 435
16.3.1 Loop Bounds 436
16.3.2 Exponential Path Space 438
16.3.3 Path Feasibility 439
16.3.4 Memory Hierarchy 440
16.4 Basics of Execution Time Analysis 442
16.4.1 Optimization Formulation 442
16.4.2 Logical Flow Constraints 445
16.4.3 Bounds for Basic Blocks 449
16.5 Other Quantitative Analysis Problems 451
16.5.1 Memory-bound Analysis 451
16.5.2 Power and Energy Analysis 452
16.6 Summary 452
Sidebar: Tools for Execution-Time Analysis 454
Exercises 455

Will my brake-by-wire system actuate the brakes within one millisecond? Answering this question requires, in part, an **execution-time analysis** of the software that runs on the electronic control unit (ECU) for the brake-by-wire system. Execution time of the software is an example of a **quantitative property** of an embedded system. The constraint that the system actuate the brakes within one millisecond is a **quantitative constraint**. The analysis of quantitative properties for conformance with quantitative constraints is central to the correctness of embedded systems and is the topic of the present chapter.

A quantitative property of an embedded system is any property that can be measured. This includes physical parameters, such as position or velocity of a vehicle controlled by the embedded system, weight of the system, operating temperature, power consumption, or reaction time. Our focus in this chapter is on properties of software-controlled systems, with particular attention to execution time. We present program analysis techniques that can ensure that execution time constraints will be met. We also discuss how similar techniques can be used to analyze other quantitative properties of software, particularly resource usage such as power, energy, and memory.

The analysis of quantitative properties requires adequate models of both the software components of the system and of the environment in which the software executes. The environment includes the processor, operating system, input-output devices, physical components with which the software interacts, and (if applicable) the communication network. The environment is sometimes also referred to as the platform on which the software executes. Providing a comprehensive treatment of execution time analysis would require much more than one chapter. The goal of this chapter is more modest. We illustrate key features of programs and their environment that must be considered in quantitative analysis, and we describe qualitatively some analysis techniques that are used. For concreteness, we focus on a single quantity, *execution time*, and only briefly discuss other resource-related quantitative properties.

16.1 Problems of Interest

The typical quantitative analysis problem involves a software task defined by a program P, the environment E in which the program executes, and the quantity of interest q. We assume that q can be given by a function of f_P as follows,

$$q = f_P(x, w)$$

where x denotes the inputs to the program P (such as data read from memory or from sensors, or data received over a network), and w denotes the environment parameters (such as network delays or the contents of the cache when the program begins executing). Defining the function f_P completely is often neither feasible nor necessary; instead, practical quantitative analysis will yield extreme values for q (highest or lowest values), average values for q, or proofs that q satisfies certain threshold constraints. We elaborate on these next.

16.1.1 Extreme-Case Analysis

In extreme-case analysis, we may want to estimate the *largest value* of q for all values of x and w,

$$\max_{x,w} \; f_P(x, w). \tag{16.1}$$

Alternatively, it can be useful to estimate the *smallest value* of q:

$$\min_{x,w} \; f_P(x, w). \tag{16.2}$$

If q represents execution time of a program or a program fragment, then the largest value is called the **worst-case execution time** (**WCET**), and the smallest value is called the **best-case execution time** (**BCET**). It may be difficult to determine these numbers exactly, but for many applications, an upper bound on the WCET or a lower bound on the BCET is all that is needed. In each case, when the computed bound equals the actual WCET or BCET, it is said to be a **tight bound**; otherwise, if there is a considerable gap between the actual value and the computed bound, it is said to be a **loose bound**. Computing loose bounds may be much easier than finding tight bounds.

16.1.2 Threshold Analysis

A **threshold property** asks whether the quantity q is always bounded above or below by a threshold T, for any choice of x and w. Formally, the property can be expressed as

$$\forall x, w, \quad f_P(x, w) \leq T \tag{16.3}$$

or

$$\forall x, w, \quad f_P(x, w) \geq T \tag{16.4}$$

Threshold analysis may provide assurances that a quantitative constraint is met, such as the requirement that a brake-by-wire system actuate the brakes within one millisecond.

Threshold analysis may be easier to perform than extreme-case analysis. Unlike extreme-case analysis, threshold analysis does not require us to determine the maximum or minimum value exactly, or even to find a tight bound on these values. Instead, the analysis is provided some guidance in the form of the target value T. Of course, it might be possible to use extreme-case analysis to check a threshold property. Specifically, Constraint 16.3 holds if the WCET does not exceed T, and Constraint 16.4 holds if the BCET is not less than T.

16.1.3 Average-Case Analysis

Often one is interested more in typical resource usage rather than in worst-case scenarios. This is formalized as average-case analysis. Here, the values of input x and environment parameter w are assumed to be drawn randomly from a space of possible values X and W according to probability distributions $\mathcal{D}_x$ and $\mathcal{D}_w$ respectively. Formally, we seek to estimate the value

$$\mathbb{E}_{\mathcal{D}_x,\mathcal{D}_w} f_P(x, w) \tag{16.5}$$

where $\mathbb{E}_{\mathcal{D}_x,\mathcal{D}_w}$ denotes the expected value of $f_P(x, w)$ over the distributions $\mathcal{D}_x$ and $\mathcal{D}_w$.

One difficulty in average-case analysis is to define realistic distributions $\mathcal{D}_x$ and $\mathcal{D}_w$ that capture the true distribution of inputs and environment parameters with which a program will execute.

In the rest of this chapter, we will focus on a single representative problem, namely, WCET estimation.

16.2 Programs as Graphs

A fundamental abstraction used often in program analysis is to represent a program as a graph indicating the flow of control from one code segment to another. We will illustrate this abstraction and other concepts in this chapter using the following running example:

Example 16.1: Consider the function `modexp` that performs **modular exponentiation**, a key step in many cryptographic algorithms. In modular exponentiation, given a base b, an exponent e, and a modulus m, one must compute b^e mod m. In the program below, `base`, `exponent` and `mod` represent b, e and m respectively. `EXP_BITS` denotes the number of bits in the exponent. The function uses a standard shift-square-accumulate algorithm, where the base is repeatedly squared, once for each bit position of the exponent, and the base is accumulated into the result only if the corresponding bit is set.

```
1  #define EXP_BITS 32
2
3  typedef unsigned int UI;
4
5  UI modexp(UI base, UI exponent, UI mod) {
6    int i;
7    UI result = 1;
8
9    i = EXP_BITS;
10   while(i > 0) {
11     if ((exponent & 1) == 1) {
12       result = (result * base) % mod;
13     }
14     exponent >>= 1;
15     base = (base * base) % mod;
16     i--;
17   }
18   return result;
19 }
```

16.2.1 Basic Blocks

A **basic block** is a sequence of consecutive program statements in which the flow of control enters only at the beginning of this sequence and leaves only at the end, without halting or the possibility of branching except at the end.

Example 16.2: The following three statements from the `modexp` function in Example 16.1 form a basic block:

```
14      exponent >>= 1;
15      base = (base * base) % mod;
16      i--;
```

Another example of a basic block includes the initializations at the top of the function, comprising lines 7 and 9:

```
7   result = 1;
8
9   i = EXP_BITS;
```

16.2.2 Control-Flow Graphs

A **control-flow graph** (**CFG**) of a program P is a directed graph $G = (V, E)$, where the set of vertices V comprises basic blocks of P, and the set of edges E indicates the flow of control between basic blocks. Figure 16.1 depicts the CFG for the `modexp` program of Example 16.1. Each node of the CFG is labeled with its corresponding basic block. In most cases, this is simply the code as it appears in Example 16.1. The only exception is for conditional statements, such as the conditions in `while` loops and `if` statements; in these cases, we follow the convention of labeling the node with the condition followed by a question mark to indicate the conditional branch.

Although our illustrative example of a control-flow graph is at the level of C source code, it is possible to use the CFG representation at other levels of program representation as well, including a high-level model as well as low-level assembly code. The level of representation employed depends on the level of detail required by the context. To make them easier to follow, our control-flow graphs will be at the level of source code.

16.2.3 Function Calls

Programs are typically decomposed into several functions in order to systematically organize the code and promote reuse and readability. The control-flow graph (CFG) representation can be extended to reason about code with function calls by introducing special

call and **return** edges. These edges connect the CFG of the **caller function** – the one making the function call – to that of the **callee function** – the one being called. A **call edge** indicates a transfer of control from the caller to the callee. A **return edge** indicates a transfer of control from the callee back to the caller.

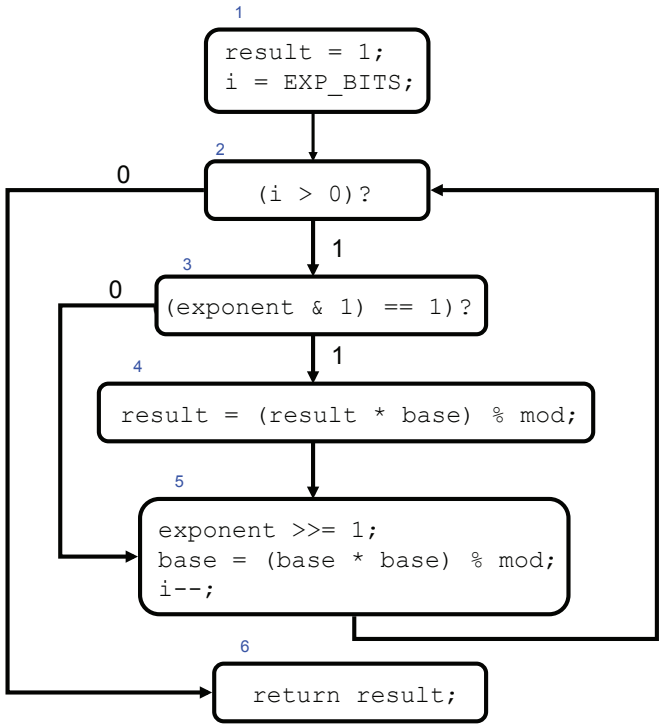


Figure 16.1: Control-flow graph for the `modexp` function of Example 16.1. All incoming edges at a node indicate transfer of control to the start of the basic block for that node, and all outgoing edges from a node indicate an exit from the end of the basic block for that node. For clarity, we label the outgoing edges from a branch statement with 0 or 1 indicating the flow of control in case the branch evaluates to false or true, respectively. An ID number for each basic block is noted above the node for that block; IDs range from 1 to 6 for this example.

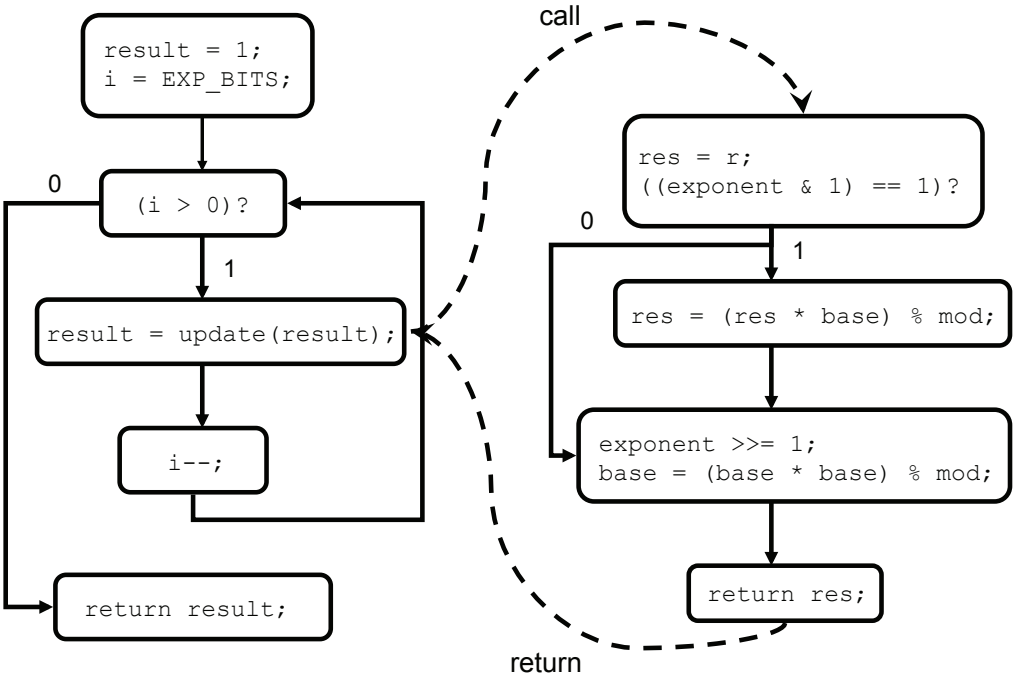


Figure 16.2: Control-flow graphs for the `modexp_call` and `update` functions in Example 16.3. Call/return edges are indicated with dashed lines.

Example 16.3: A slight variant shown below of the modular exponentation program of Example 16.1 uses function calls and can be represented by the CFG with call and return edges in Figure 16.2.

```
1  #define EXP_BITS 32
2  typedef unsigned int UI;
3  UI exponent, base, mod;
4
5  UI update(UI r) {
6    UI res = r;
7    if ((exponent & 1) == 1) {
8      res = (res * base) % mod;
9    }
```

```
10    exponent >>= 1;
11    base = (base * base) % mod;
12    return res;
13  }
14
15  UI modexp_call() {
16    UI result = 1; int i;
17    i = EXP_BITS;
18    while(i > 0) {
19      result = update(result);
20      i--;
21    }
22    return result;
23  }
```

In this modified example, the variables `base`, `exponent`, and `mod` are global variables. The update to `base` and `exponent` in the body of the `while` loop, along with the computation of `result` is now performed in a separate function named `update`.

Non-recursive function calls can also be handled by **inlining**, which is the process of copying the code for the callee into that of the caller. If inlining is performed transitively for all functions called by the code that must be analyzed, the analysis can be performed on the CFG of the code resulting from inlining, without using call and return edges.

16.3 Factors Determining Execution Time

There are several issues one must consider in order to estimate the worst-case execution time of a program. This section outlines some of the main issues and illustrates them with examples. In describing these issues, we take a programmer's viewpoint, starting with the program structure and then considering how the environment can impact the program's execution time.

16.3.1 Loop Bounds

The first point one must consider when bounding the execution time of a program is whether the program terminates. Non-termination of a sequential program can arise from non-terminating loops or from an unbounded sequence of function calls. Therefore, while writing real-time embedded software, the programmer must ensure that all loops are guaranteed to terminate. In order to guarantee this, one must determine for each loop a bound on the number of times that loop will execute in the worst case. Similarly, all function calls must have bounded recursion depth. The problems of determining bounds on loop iterations or recursion depth are undecidable in general, since the halting problem for Turing machines can be reduced to either problem. (See Appendix B for an introduction to Turing machines and decidability.)

In this section, we limit ourselves to reasoning about loops. In spite of the undeciable nature of the problem, progress has been made on automatically determining loop bounds for several patterns that arise in practice. Techniques for determining loop bounds are a current research topic and a full survey of these methods is out of the scope of this chapter. We will limit ourselves to presenting illustrative examples for loop bound inference.

The simplest case is that of `for` loops that have a specified constant bound, as in Example 16.4 below. This case occurs often in embedded software, in part due to a discipline of programming enforced by designers who must program for real-time constraints and limited resources.

Example 16.4: Consider the function `modexp1` below. It is a slight variant of the function `modexp` introduced in Example 16.1 that performs modular exponentiation, in which the `while` loop has been expressed as an equivalent `for` loop.

```
1  #define EXP_BITS 32
2
3  typedef unsigned int UI;
4
5  UI modexp1(UI base, UI exponent, UI mod) {
6    UI result = 1; int i;
7
8    for(i=EXP_BITS; i > 0; i--) {
9      if ((exponent & 1) == 1) {
10       result = (result * base) % mod;
```

```
11      }
12      exponent >>= 1;
13      base = (base * base) % mod;
14    }
15    return result;
16  }
```

In the case of this function, it is easy to see that the `for` loop will take exactly `EXP_BITS` iterations, where `EXP_BITS` is defined as the constant 32.

In many cases, the loop bound is not immediately obvious (as it was for the above example). To make this point, here is a variation on Example 16.4.

Example 16.5: The function listed below also performs modular exponentiation, as in Example 16.4. However, in this case, the `for` loop is replaced by a `while` loop with a different loop condition – the loop exits when the value of `exponent` reaches 0. Take a moment to check whether the `while` loop will terminate (and if so, why).

```
1  typedef unsigned int UI;
2
3  UI modexp2(UI base, UI exponent, UI mod) {
4    UI result = 1;
5
6    while (exponent != 0) {
7      if ((exponent & 1) == 1) {
8        result = (result * base) % mod;
9      }
10     exponent >>= 1;
11     base = (base * base) % mod;
12   }
13   return result;
14 }
```

Now let us analyze the reason that this loop terminates. Notice that `exponent` is an unsigned int, which we will assume to be 32 bits wide. If it starts out equal to 0, the loop terminates right away and the function returns `result = 1`. If not, in each iteration of the loop, notice that line 10 shifts `exponent` one bit

to the right. Since `exponent` is an unsigned int, after the right shift, its most significant bit will be 0. Reasoning thus, after at most 32 right shifts, all bits of `exponent` must be set to 0, thus causing the loop to terminate. Therefore, we can conclude that the loop bound is 32.

Let us reflect on the reasoning employed in the above example. The key component of our "proof of termination" was the observation that the number of bits of `exponent` decreases by 1 each time the loop executes. This is a standard argument for proving termination – by defining a **progress measure** or **ranking function** that maps each state of the program to a mathematical structure called a **well order**. Intuitively, a well order is like a program that counts down to zero from some initial value in the natural numbers.

16.3.2 Exponential Path Space

Execution time is a path property. In other words, the amount of time taken by the program is a function of how conditional statements in the program evaluate to true or false. A major source of complexity in execution time analysis (and other program analysis problems as well) is that the number of program paths can be very large — exponential in the size of the program. We illustrate this point with the example below.

Example 16.6: Consider the function `count` listed below, which runs over a two-dimensional array, counting and accumulating non-negative and negative elements of the array separately.

```
1  #define MAXSIZE 100
2
3  int Array[MAXSIZE][MAXSIZE];
4  int Ptotal, Pcnt, Ntotal, Ncnt;
5  ...
6  void count() {
7    int Outer, Inner;
8    for (Outer = 0; Outer < MAXSIZE; Outer++) {
9      for (Inner = 0; Inner < MAXSIZE; Inner++)  {
10       if (Array[Outer][Inner] >= 0) {
11         Ptotal += Array[Outer][Inner];
```

```
12            Pcnt++;
13          } else {
14            Ntotal += Array[Outer][Inner];
15            Ncnt++;
16          }
17        }
18      }
19    }
```

The function includes a nested loop. Each loop executes `MAXSIZE` (100) times. Thus, the inner body of the loop (comprising lines 10–16) will execute 10,000 times – as many times as the number of elements of `Array`. In each iteration of the inner body of the loop, the conditional on line 10 can either evaluate to true or false, thus resulting in 2^{10000} possible ways the loop can execute. In other words, this program has 2^{10000} paths.

Fortunately, as we will see in Section 16.4.1, one does not need to explicitly enumerate all possible program paths in order to perform execution time analysis.

16.3.3 Path Feasibility

Another source of complexity in program analysis is that all program paths may not be executable. A computationally expensive function is irrelevant for execution time analysis if that function is never executed.

A path p in program P is said to be **feasible** if there exists an input x to P such that P executes p on x. In general, even if P is known to terminate, determining whether a path p is feasible is a computationally intractable problem. One can encode the canonical NP-complete problem, the Boolean satisfiability problem (see Appendix B), as a problem of checking path feasibility in a specially-constructed program. In practice, however, in many cases, it is possible to determine path feasibility.

Example 16.7: Recall Example 13.3 of a software task from the open source Paparazzi unmanned aerial vehicle (UAV) project (Nemer et al., 2006):

```
1  #define PPRZ_MODE_AUTO2 2
2  #define PPRZ_MODE_HOME 3
3  #define VERTICAL_MODE_AUTO_ALT 3
4  #define CLIMB_MAX 1.0
5  ...
6  void altitude_control_task(void) {
7    if (pprz_mode == PPRZ_MODE_AUTO2
8        || pprz_mode == PPRZ_MODE_HOME) {
9      if (vertical_mode == VERTICAL_MODE_AUTO_ALT) {
10       float err = estimator_z - desired_altitude;
11       desired_climb
12             = pre_climb + altitude_pgain * err;
13       if (desired_climb < -CLIMB_MAX) {
14         desired_climb = -CLIMB_MAX;
15       }
16       if (desired_climb > CLIMB_MAX) {
17         desired_climb = CLIMB_MAX;
18       }
19     }
20   }
21 }
```

This program has 11 paths in all. However, the number of *feasible* program paths is only 9. To see this, note that the two conditionals `desired_climb < -CLIMB_MAX` on line 13 and `desired_climb > CLIMB_MAX` on line 16 cannot both be true. Thus, only three out of the four paths through the two innermost conditional statements are feasible. This infeasible inner path can be taken for two possible evaluations of the outermost conditional on lines 7 and 8: either if `pprz_mode == PPRZ_MODE_AUTO2` is true, or if that condition is false, but `pprz_mode == PPRZ_MODE_HOME` is true.

16.3.4 Memory Hierarchy

The preceding sections have focused on properties of programs that affect execution time. We now discuss how properties of the execution platform, specifically of cache memories, can significantly impact execution time. We illustrate this point using Example 16.8.[1] The material on caches introduced in Sec. 9.2.3 is pertinent to this discussion.

[1] This example is based on a similar example in Bryant and O'Hallaron (2003).

Example 16.8: Consider the function `dot_product` listed below, which computes the dot product of two vectors of floating point numbers. Each vector is of dimension `n`, where `n` is an input to the function. The number of iterations of the loop depends on the value of `n`. However, even if we know an upper bound on `n`, hardware effects can still cause execution time to vary widely for similar values of `n`.

```
1 float dot_product(float *x, float *y, int n) {
2   float result = 0.0;
3   int i;
4   for(i=0; i < n; i++) {
5     result += x[i] * y[i];
6   }
7   return result;
8 }
```

Suppose this program is executing on a 32-bit processor with a direct-mapped cache. Suppose also that the cache can hold two sets, each of which can hold 4 `floats`. Finally, let us suppose that `x` and `y` are stored contiguously in memory starting with address 0.

Let us first consider what happens if `n` = 2. In this case, the entire arrays `x` and `y` will be in the same block and thus in the same cache set. Thus, in the very first iteration of the loop, the first access to read `x[0]` will be a cache miss, but thereafter every read to `x[i]` and `y[i]` will be a cache hit, yielding best case performance for loads.

Consider next what happens when `n` = 8. In this case, each `x[i]` and `y[i]` map to the same cache set. Thus, not only will the first access to `x[0]` be a miss, the first access to `y[0]` will also be a miss. Moreover, the latter access will evict the block containing `x[0]`-`x[3]`, leading to a cache miss on `x[1]`, `x[2]`, and `x[3]` as well. The reader can see that every access to an `x[i]` or `y[i]` will lead to a cache miss.

Thus, a seemingly small change in the value of `n` from 2 to 8 can lead to a drastic change in execution time of this function.

16.4 Basics of Execution Time Analysis

Execution time analysis is a current research topic, with many problems still to be solved. There have been over two decades of research, resulting in a vast literature. We cannot provide a comprehensive survey of the methods in this chapter. Instead, we will present some of the basic concepts that find widespread use in current techniques and tools for WCET analysis. Readers interested in a more detailed treatment may find an overview in a recent survey paper (Wilhelm et al., 2008) and further details in books (e.g., Li and Malik (1999)) and book chapters (e.g., Wilhelm (2005)).

16.4.1 Optimization Formulation

An intuitive formulation of the WCET problem can be constructed using the view of programs as graphs. Given a program P, let $G = (V, E)$ denote its control-flow graph (CFG). Let $n = |V|$ be the number of nodes (basic blocks) in G, and $m = |E|$ denote the number of edges. We refer to the basic blocks by their index i, where i ranges from 1 to n.

We assume that the CFG has a unique *start* or *source* node s and a unique *sink* or *end* node t. This assumption is not restrictive: If there are multiple start or end nodes, one can add a dummy start/end node to achieve this condition. Usually we will set $s = 1$ and $t = n$.

Let x_i denote the number of times basic block i is executed. We call x_i the **execution count** of basic block i. Let $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ be a vector of variables recording execution counts. Not all valuations of $\mathbf{x}$ correspond to valid program executions. We say that $\mathbf{x}$ is **valid** if the elements of $\mathbf{x}$ correspond to a (valid) execution of the program. The following example illustrates this point.

Example 16.9: Consider the CFG for the modular exponentiation function `modexp` introduced in Example 16.1. There are six basic blocks in this function, labeled 1 to 6 in Figure 16.1. Thus, $\mathbf{x} = (x_1, x_2, \ldots, x_6)$. Basic blocks 1 and 6, the start and end, are each executed only once. Thus, $x_1 = x_6 = 1$; any other valuation cannot correspond to any program execution.

Next consider basic blocks 2 and 3, corresponding to the conditional branches `i > 0` and `(exponent & 1) == 1`. One can observe that x_2 must equal $x_3 + 1$, since the block 3 is executed every time block 2 is executed, except when the loop exits to block 6.

Along similar lines, one can see that basic blocks 3 and 5 must be executed an equal number of times.

Flow Constraints

The intuition expressed in Example 16.9 can be formalized using the theory of **network flow**, which finds use in many contexts including modeling traffic, fluid flow, and the flow of current in an electrical circuit. In particular, in our problem context, the flow must satisfy the following two properties:

1. *Unit Flow at Source:* The control flow from source node $s = 1$ to sink node $t = n$ is a single execution and hence corresponds to unit flow from source to sink. This property is captured by the following two constraints:

$$x_1 = 1 \tag{16.6}$$
$$x_n = 1 \tag{16.7}$$

2. *Conservation of Flow:* For each node (basic block) i, the incoming flow to i from its predecessor nodes equals the outgoing flow from i to its successor nodes.

   To capture this property, we introduce additional variables to record the number of times that each edge in the CFG is executed. Following the notation of Li and Malik (1999), let d_{ij} denote the number of times the edge from node i to node j in the CFG is executed. Then we require that for each node i, $1 \leq i \leq n$,

$$x_i = \sum_{j \in P_i} d_{ji} = \sum_{j \in S_i} d_{ij}, \tag{16.8}$$

   where P_i is the set of predecessors to node i and S_i is the set of successors. For the source node, $P_1 = \emptyset$, so the sum over predecessor nodes is omitted. Similarly, for the sink node, $S_n = \emptyset$, so the sum over successor nodes is omitted.

Taken together, the two sets of constraints presented above suffice to implicitly define all source-to-sink execution paths of the program. Since this constraint-based representation is an *implicit* representation of program paths, this approach is also referred to in the literature as **implicit path enumeration** or **IPET**.

We illustrate the generation of the above constraints with an example.

Example 16.10: Consider again the function `modexp` of Example 16.1, with CFG depicted in Figure 16.1.

The constraints for this CFG are as follows:

$$\begin{aligned}
x_1 &= 1 \\
x_6 &= 1 \\
x_1 &= d_{12} \\
x_2 &= d_{12} + d_{52} = d_{23} + d_{26} \\
x_3 &= d_{23} = d_{34} + d_{35} \\
x_4 &= d_{34} = d_{45} \\
x_5 &= d_{35} + d_{45} = d_{52} \\
x_6 &= d_{26}
\end{aligned}$$

Any solution to the above system of equations will result in integer values for the x_i and d_{ij} variables. Furthermore, this solution will generate valid execution counts for basic blocks. For example, one such valid solution is

$$\begin{aligned}
&x_1 = 1,\ d_{12} = 1,\ x_2 = 2,\ d_{23} = 1,\ x_3 = 1,\ d_{34} = 0,\ d_{35} = 1, \\
&x_4 = 0,\ d_{45} = 0,\ x_5 = 1,\ d_{52} = 1,\ x_6 = 1,\ d_{26} = 1.
\end{aligned}$$

Readers are invited to find and examine additional solutions for themselves.

Overall Optimization Problem

We are now in a position to formulate the overall optimization problem to determine worst-case execution time. The key assumption we make in this section is that we know

an upper bound w_i on the execution time of the basic block i. (We will later see in Section 16.4.3 how the execution time of a single basic block can be bounded.) Then the WCET is given by the maximum $\sum_{i=1}^{n} w_i x_i$ over valid execution counts x_i.

Putting this together with the constraint formulation of the preceding section, our goal is to find values for x_i that give

$$\max_{x_i,\, 1 \le i \le n} \sum_{i=1}^{n} w_i x_i$$

subject to

$$\begin{aligned} x_1 &= x_n = 1 \\ x_i &= \sum_{j \in P_i} d_{ji} = \sum_{j \in S_i} d_{ij} \end{aligned}$$

This optimization problem is a form of a **linear programming** (**LP**) problem (also called a **linear program**), and it is solvable in polynomial time.

However, two major challenges remain:

- This formulation assumes that all source to sink paths in the CFG are feasible and does not bound loops in paths. As we have already seen in Section 16.3, this is not the case in general, so solving the above maximization problem may yield a pessimistic loose bound on the WCET. We will consider this challenge in Section 16.4.2.
- The upper bounds w_i on execution time of basic blocks i are still to be determined. We will briefly review this topic in Section 16.4.3.

16.4.2 Logical Flow Constraints

In order to ensure that the WCET optimization is not too pessimistic by including paths that cannot be executed, we must add so-called **logical flow constraints**. These constraints rule out infeasible paths and incorporate bounds on the number of loop iterations. We illustrate the use of such constraints with two examples.

Loop Bounds

For programs with loops, it is necessary to use bounds on loop iterations to bound execution counts of basic blocks.

Example 16.11: Consider the modular exponentiation program of Example 16.1 for which we wrote down flow constraints in Example 16.10.

Notice that those constraints impose no upper bound on x_2 or x_3. As argued in Examples 16.4 and 16.5, the bound on the number of loop iterations in this example is 32. However, without imposing this additional constraint, since there is no upper bound on x_2 or x_3, the solution to our WCET optimization will be infinite, implying that there is no upper bound on the WCET. The following single constraint suffices:

$$x_3 \leq 32$$

From this constraint on x_3, we derive the constraint that $x_2 \leq 33$, and also upper bounds on x_4 and x_5. The resulting optimization problem will then return a finite solution, for finite values of w_i.

Adding such bounds on values of x_i does not change the complexity of the optimization problem. It is still a linear programming problem.

Infeasible Paths

Some logical flow constraints rule out combinations of basic blocks that cannot appear together on a single path.

Example 16.12: Consider a snippet of code from Example 16.7 describing a software task from the open source Paparazzi unmanned aerial vehicle (UAV) project (Nemer et al., 2006):

```
1 #define CLIMB_MAX 1.0
```

```
2  ...
3  void altitude_control_task(void) {
4     ...
5     err = estimator_z - desired_altitude;
6     desired_climb
7           = pre_climb + altitude_pgain * err;
8     if (desired_climb < -CLIMB_MAX) {
9       desired_climb = -CLIMB_MAX;
10    }
11    if (desired_climb > CLIMB_MAX) {
12      desired_climb = CLIMB_MAX;
13    }
14    return;
15 }
```

The CFG for the snippet of code shown above is given in Figure 16.3. The system of flow constraints for this CFG according to the rules in Section 16.4.1 is as follows:

$$\begin{aligned} x_1 &= 1 \\ x_5 &= 1 \\ x_1 &= d_{12} + d_{13} \\ x_2 &= d_{12} = d_{23} \\ x_3 &= d_{13} + d_{23} = d_{34} + d_{35} \\ x_4 &= d_{34} = d_{45} \\ x_5 &= d_{35} + d_{45} \end{aligned}$$

A solution for the above system of equations is

$$x_1 = x_2 = x_3 = x_4 = x_5 = 1,$$

implying that each basic block gets executed exactly once, and that both conditionals evaluate to *true*. However, as we discussed in Example 16.7, it is impossible for both conditionals to evaluate to *true*. Since `CLIMB_MAX` = 1.0, if `desired_climb` is less than -1.0 in basic block 1, then at the start of basic block 3 it will be set to -1.0.

The following constraint rules out the infeasible path:

$$d_{12} + d_{34} \leq 1 \tag{16.9}$$

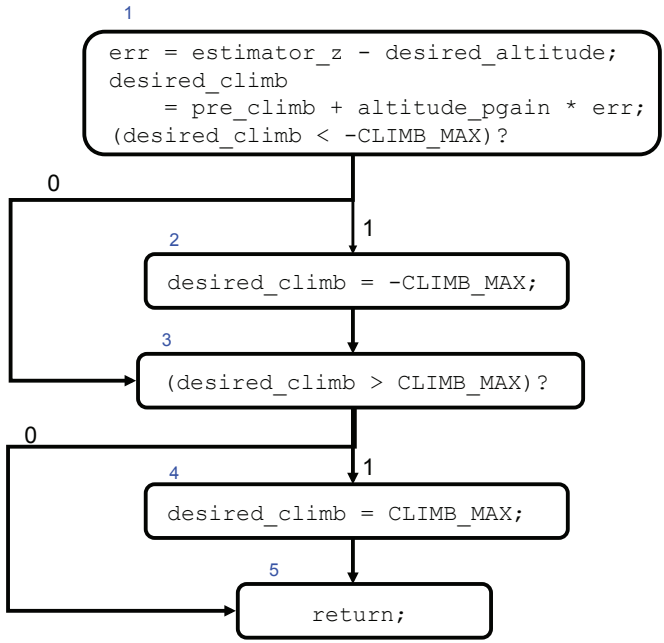


Figure 16.3: Control-flow graph for Example 16.12.

This constraint specifies that both conditional statements cannot be *true* together. It is of course possible for both conditionals to be *false*. We can check that this constraint excludes the infeasible path when added to the original system.

More formally, for a program *without loops*, if a set of k edges

$$(i_1, j_1), (i_2, j_2), \ldots, (i_k, j_k)$$

in the CFG cannot be taken together in a program execution, the following constraint is added to the optimization problem:

$$d_{i_1 j_1} + d_{i_2 j_2} + \ldots + d_{i_k j_k} \leq k - 1 \tag{16.10}$$

For programs with loops, the constraint is more complicated since an edge can be traversed multiple times, so the value of a d_{ij} variable can exceed 1. We omit the details in

this case; the reader can consult Li and Malik (1999) for a more elaborate discussion of this topic.

In general, the constraints added above to exclude infeasible combinations of edges can change the complexity of the optimization problem, since one must also add the following **integrality** constraints:

$$x_i \in \mathbb{N}, \quad \text{for all } i = 1, 2, \ldots, n \tag{16.11}$$

$$d_{ij} \in \mathbb{N}, \quad \text{for all } i, j = 1, 2, \ldots, n \tag{16.12}$$

In the absence of such integrality constraints, the optimization solver can return fractional values for the x_i and d_{ij} variables. However, adding these constraints results in an integer linear programming (ILP) problem. The ILP problem is known to be NP-hard (see Appendix B, Section B.4). Even so, in many practical instances, one can solve these ILP problems fairly efficiently (see for example Li and Malik (1999)).

16.4.3 Bounds for Basic Blocks

In order to complete the optimization problem for WCET analysis, we need to compute upper bounds on the execution times of basic blocks – the w_i coefficients in the cost function of Section 16.4.1. Execution time is typically measured in CPU cycles. Generating such bounds requires detailed microarchitectural modeling. We briefly outline some of the issues in this section.

A simplistic approach to this problem would be to generate conservative upper bounds on the execution time of each instruction in the basic block, and then add up these per-instruction bounds to obtain an upper bound on the execution time of the overall basic block.

The problem with this approach is that there can be very wide variation in the execution times for some instructions, resulting in very loose upper bounds on the execution time of a basic block. For instance, consider the latency of memory instructions (loads and stores) for a system with a data cache. The difference between the latency when there is a cache miss versus a hit can be a factor of 100 on some platforms. In these cases, if the analysis does not differentiate between cache hits and misses, it is possible for the computed bound to be a hundred times larger than the execution time actually exhibited.

Several techniques have been proposed to better use program context to predict execution time of instructions more precisely. These techniques involve detailed microarchitectural modeling. We mention two main approaches below:

- *Integer linear programming (ILP) methods:* In this approach, pioneered by Li and Malik (1999), one adds **cache constraints** to the ILP formulation of Section 16.4.1. Cache constraints are linear expressions used to bound the number of cache hits and misses within basic blocks. The approach tracks the memory locations that cause *cache conflicts* – those that map onto the same cache set, but have different tags – and adds linear constraints to record the impact of such conflicts on the number of cache hits and misses. Measurement through simulation or execution on the actual platform must be performed to obtain the cycle count for hits and misses. The cost constraint of the ILP is modified to compute the program path along which the overall number of cycles, including cache hits and misses, is the largest. Further details about this approach are available in Li and Malik (1999).

- *Abstract interpretation methods:* **Abstract interpretation** is a theory of approximation of mathematical structures, in particular those that arise in defining the semantic models of computer systems (Cousot and Cousot (1977)). In particular, in abstract interpretation, one performs **sound approximation**, where the set of behaviors of the system is a subset of that of the model generated by abstract interpretation. In the context of WCET analysis, abstract interpretation has been used to infer invariants at program points, in order to generate loop bounds, and constraints on the state of processor pipelines or caches at the entry and exit locations of basic blocks. For example, such a constraint could specify the conditions under which variables will be available in the data cache (and hence a cache hit will result). Once such constraints are generated, one can run measurements from states satisfying those constraints in order to generate execution time estimates. Further details about this approach can be found in Wilhelm (2005).

In addition to techniques such as those described above, accurate measurement of execution time is critical for finding tight WCET bounds. Some of the measurement techniques are as follows:

1. *Sampling CPU cycle counter:* Certain processors include a register that records the number of CPU cycles elapsed since reset. For example, the **time stamp counter register** on x86 architectures performs this function, and is accessible through a

`rdtsc` ("read time stamp counter") instruction. However, with the advent of multi-core designs and power management features, care must be taken to use such CPU cycle counters to accurately measure timing. For example, it may be necessary to lock the process to a particular CPU.

2. *Using a logic analyzer:* A **logic analyzer** is an electronic instrument used to measure signals and track events in a digital system. In the current context, the events of interest are the entry and exit points of the code to be timed, definable, for example, as valuations of the program counter. Logic analyzers are less intrusive than using cycle counters, since they do not require instrumenting the code, and they can be more accurate. However, the measurement setup is more complicated.

3. *Using a cycle-accurate simulator:* In many cases, timing analysis must be performed when the actual hardware is not yet available. In this situation, a cycle-accurate simulator of the platform provides a good alternative.

16.5 Other Quantitative Analysis Problems

Although we have focused mainly on execution time in this chapter, several other quantitative analysis problems are relevant for embedded systems. We briefly describe two of these in this section.

16.5.1 Memory-bound Analysis

Embedded computing platforms have very limited memory as compared to general-purpose computers. For example, as mentioned in Chapter 9, the Luminary Micro LM3S8962 controller has only 64 KB of RAM. It is therefore essential to structure the program so that it uses memory efficiently. Tools that analyze memory consumption and compute bounds on memory usage can be very useful.

There are two kinds of memory bound analysis that are relevant for embedded systems. In **stack size analysis** (or simply **stack analysis**), one needs to compute an upper bound on the amount of stack-allocated memory used by a program. Recall from Section 9.3.2 that stack memory is allocated whenever a function is called or an interrupt is handled. If the program exceeds the memory allocated for the stack, a stack overflow is said to occur.

If the program does not contain recursive functions and runs uninterrupted, one can bound stack usage by traversing the **call graph** of the program – the graph that tracks which functions call which others. If the space for each stack frame is known, then one can track the sequence of calls and returns along paths in the call graph in order to compute the worst-case stack size.

Performing stack size analysis for interrupt-driven software is significantly more complicated. We point the interested reader to Brylow et al. (2001).

Heap analysis is the other memory bound analysis problem that is relevant for embedded systems. This problem is harder than stack bound analysis since the amount of heap space used by a function might depend on the values of input data and may not be known prior to run-time. Moreover, the exact amount of heap space used by a program can depend on the implementation of dynamic memory allocation and the garbage collector.

16.5.2 Power and Energy Analysis

Power and energy consumption are increasingly important factors in embedded system design. Many embedded systems are autonomous and limited by battery power, so a designer must ensure that the task can be completed within a limited energy budget. Also, the increasing ubiquity of embedded computing is increasing its energy footprint, which must be reduced for sustainable development.

To first order, the energy consumed by a program running on an embedded device depends on its execution time. However, estimating execution time alone is not sufficient. For example, energy consumption depends on circuit switching activity, which can depend more strongly on the data values with which instructions are executed.

For this reason, most techniques for energy and power estimation of embedded software focus on estimating the average-case consumption. The average case is typically estimated by profiling instructions for several different data values, guided by software benchmarks. For an introduction to this topic, see Tiwari et al. (1994).

16.6 Summary

Quantitative properties, involving physical parameters or specifying resource constraints, are central to embedded systems. This chapter gave an introduction to basic concepts in

quantitative analysis. First, we considered various types of quantitative analysis problems, including extreme-case analysis, average-case analysis, and verifying threshold properties. As a representative example, this chapter focused on execution time analysis. Several examples were presented to illustrate the main issues, including loop bounds, path feasibility, path explosion, and cache effects. An optimization formulation that forms the backbone of execution time analysis was presented. Finally, we briefly discussed two other quantitative analysis problems, including computing bounds on memory usage and on power or energy consumption.

Quantitative analysis remains an active field of research – exemplifying the challenges in bridging the cyber and physical aspects of embedded systems.

Tools for Execution-Time Analysis

Current techniques for execution-time analysis are broadly classified into those primarily based on **static analysis** and those that are **measurement-based**.

Static tools rely on abstract interpretation and **dataflow analysis** to compute facts about the program at selected program locations. These facts are used to identify dependencies between code fragments, generate loop bounds, and identify facts about the platform state, such as the state of the cache. These facts are used to guide timing measurements of basic blocks and combined into an optimization problem as presented in this chapter. Static tools aim to find conservative bounds on extreme-case execution time; however, they are not easy to port to new platforms, often requiring several man-months of effort.

Measurement-based tools are primarily based on testing the program on multiple inputs and then estimating the quantity of interest (e.g., WCET) from those measurements. Static analysis is often employed in performing a guided exploration of the space of program paths and for test generation. Measurement-based tools are easy to port to new platforms and apply broadly to both extreme-case and average-case analysis; however, not all techniques provide guarantees for finding extreme-case execution times.

Further details about many of these tools are available in Wilhelm et al. (2008); Seshia and Rakhlin (2012). Here is a partial list of tools with links to papers and websites:

Name	Primary Type	Institution & Website/References
aiT	Static	AbsInt Angewandte Informatik GmbH (Wilhelm, 2005) http://www.absint.com/ait/
Bound-T	Static	Tidorum Ltd. http://www.bound-t.com/
Chronos	Static	National University of Singapore (Li et al., 2005) http://www.comp.nus.edu.sg/~rpembed/chronos/
Heptane	Static	IRISA Rennes http://www.irisa.fr/aces/work/heptane-demo/heptane.html
SWEET	Static	Mälardalen University http://www.mrtc.mdh.se/projects/wcet/
GameTime	Measurement	UC Berkeley Seshia and Rakhlin (2008)
RapiTime	Measurement	Rapita Systems Ltd. http://www.rapitasystems.com/
SymTA/P	Measurement	Technical University Braunschweig http://www.ida.ing.tu-bs.de/research/projects/symtap/
Vienna M./P.	Measurement	Technical University of Vienna http://www.wcet.at/

Exercises

1. This problem studies execution time analysis. Consider the C program listed below:

```
1  int arr[100];
2
3  int foo(int flag) {
4    int i;
5    int sum = 0;
6
7    if (flag) {
8      for(i=0;i<100;i++)
9        arr[i] = i;
10   }
11
12   for(i=0;i<100;i++)
13     sum += arr[i];
14
15   return sum;
16 }
```

   Assume that this program is run on a processor with data cache of size big enough that the entire array `arr` can fit in the cache.

   (a) How many paths does the function `foo` of this program have? Describe what they are.

   (b) Let T denote the execution time of the second `for` loop in the program. How does executing the first `for` loop affect the value of T? Justify your answer.

2. Consider the program given below:

```
1  void testFn(int *x, int flag) {
2    while (flag != 1) {
3      flag = 1;
4      *x = flag;
5    }
6    if (*x > 0)
7      *x += 2;
8  }
```

   In answering the questions below, assume that `x` is not `NULL`.

   (a) Draw the control-flow graph of this program. Identify the basic blocks with unique IDs starting with 1.

Note that we have added a dummy source node, numbered 0, to represent the entry to the function. For convenience, we have also introduced a dummy sink node, although this is not strictly required.

(b) Is there a bound on the number of iterations of the while loop? Justify your answer.

(c) How many total paths does this program have? How many of them are feasible, and why?

(d) Write down the system of flow constraints, including any logical flow constraints, for the control-flow graph of this program.

(e) Consider running this program uninterrupted on a platform with a data cache. Assume that the data pointed to by `x` is not present in the cache at the start of this function.

For each read/write access to `*x`, argue whether it will be a cache hit or miss.

Now, assume that `*x` is present in the cache at the start of this function. Identify the basic blocks whose execution time will be impacted by this modified assumption.

3. Consider the function `check_password` given below that takes two arguments: a user ID `uid` and candidate password `pwd` (both modeled as `int`s for simplicity). This function checks that password against a list of user IDs and passwords stored in an array, returning 1 if the password matches and 0 otherwise.

```
1  struct entry {
2    int user;
3    int pass;
4  };
5  typedef struct entry entry_t;
6
7  entry_t all_pwds[1000];
8
9  int check_password(int uid, int pwd) {
10   int i = 0;
11   int retval = 0;
12
13   while(i < 1000) {
14     if (all_pwds[i].user == uid && all_pwds[i].pass == pwd) {
15       retval = 1;
16       break;
17     }
18     i++;
19   }
20
```

```
21    return retval;
22  }
```

(a) Draw the control-flow graph of the function `check_password`. State the number of nodes (basic blocks) in the CFG. (Remember that each conditional statement is considered a single basic block by itself.)

Also state the number of paths from entry point to exit point (ignore path feasibility).

(b) Suppose the array `all_pwds` is sorted based on passwords (either increasing or decreasing order). In this question, we explore if an external client that calls `check_password` can *infer anything about the passwords* stored in `all_pwds` by repeatedly calling it and *recording the execution time* of `check_password`. Figuring out secret data from "physical" information, such as running time, is known as a *side-channel attack*.

In each of the following two cases, what, if anything, can the client infer about the passwords in `all_pwds`?

(i) The client has exactly one (uid, password) pair present in `all_pwds`

(ii) The client has NO (uid, password) pairs present in in `all_pwds`

Assume that the client knows the program but not the contents of the array `all_pwds`.

4. Consider the code below that implements the logic of a highly simplified vehicle automatic transmission system. The code aims to set the value of `current_gear` based on a sensor input `rpm`. `LO_VAL` and `HI_VAL` are constants whose exact values are irrelevant to this problem (you can assume that `LO_VAL` is strictly smaller than `HI_VAL`).

```
1   volatile float rpm;
2
3   int current_gear; // values range from 1 to 6
4
5   void change_gear() {
6     if (rpm < LO_VAL)
7       set_gear(-1);
8     else {
9       if (rpm > HI_VAL)
10        set_gear(1);
11    }
12
13    return;
```

```
14  }
15
16  void set_gear(int update) {
17    int new_gear = current_gear + update;
18    if (new_gear > 6)
19      new_gear = 6;
20    if (new_gear < 1)
21      new_gear = 1;
22
23    current_gear = new_gear;
24
25    return;
26  }
```

This is a 6-speed automatic transmission system, and thus the value of `current_gear` ranges between 1 and 6.

Answer the following questions based on the above code:

(a) Draw the control-flow graph (CFG) of the program starting from `change_gear`, *without inlining* function `set_gear`. In other words, you should draw the CFG using call and return edges.

For brevity, you need not write the code for the basic blocks inside the nodes of the CFG. Just indicate which statements go in which node by using the line numbers in the code listing above.

(b) Count the number of execution paths from the entry point in `set_gear` to its exit point (the return statement). Ignore feasibility issues for this question. Also count the number of paths from the entry point in `change_gear` to its exit point (the return statement), including the paths through `set_gear`. State the number of paths in each case.

(c) Now consider path feasibility. Recalling that `current_gear` ranges between 1 and 6, how many feasible paths does `change_gear` have? Justify your answer.

(d) Give an example of a feasible path and of an infeasible path through the function `change_gear`. Describe each path as a sequence of line numbers, ignoring the line numbers corresponding to function definitions and return statements.

17 Security and Privacy

17.1 Cryptographic Primitives 461
17.1.1 Encryption and Decryption 461
17.1.2 Digital Signatures and Secure Hash Functions 465
Sidebar: Implementation Matters for Cryptography 466
17.2 Protocol and Network Security 469
17.2.1 Key Exchange 469
Sidebar: Reverse Engineering Systems to Improve Security 472
17.2.2 Cryptographic Protocol Design 473
17.3 Software Security 474
17.4 Information Flow 477
17.4.1 Examples 478
17.4.2 Theory 482
17.4.3 Analysis and Enforcement 484
17.5 Advanced Topics 485
17.5.1 Sensor and Actuator Security 486
17.5.2 Side-channel Attacks 488
17.6 Summary 490
Exercises 491

Security and **privacy** are two of the foremost design concerns for cyber-physical systems today. Security, broadly speaking, is the state of being protected from harm. Privacy is the state of being kept away from observation. With embedded and cyber-physical systems being increasingly networked with each other and with the Internet, security and privacy concerns are now front and center for system designers.

In a formal sense, there are two primary aspects that differentiate security and privacy from other design criteria. First, the operating environment is considered to be significantly more adversarial in nature than in typical system design. Second, the kinds of properties, specifying desired and undesired behavior, are also different from traditional system specifications (and often impose additional requirements on top of the traditional ones). Let us consider each of these aspects in turn.

The notion of an **attacker** or an **adversary** is central to the theory and practice of security and privacy. An attacker is a malicious agent whose goal is to subvert the operation of the system in some way. The exact subversion depends on characteristics of the system, its goals and requirements, and the capabilities of the attacker. Typically, these characteristics are grouped together into an entity called the **threat model** or **attacker model**. For example, while designing an automobile with no wireless network connectivity, the designers may assume that the only threats arise from people who have physical access to the automobile and knowledge of its components, such as a well-trained mechanic. Transforming an informal threat model (such as the preceding sentence) into a precise, mathematical statement of an attacker's objectives is a challenging task, but one that is essential for principled model-based design of secure embedded systems.

The second defining characteristic of the field of security and privacy are the distinctive properties it is concerned with. Broadly speaking, these properties can be categorized into the following types: confidentiality, integrity, authenticity, and availability. **Confidentiality** is the state of being secret from the attacker. A good example of confidential data is a password or PIN that one uses to access one's bank account. **Integrity** is the state of being unmodified by an attacker. An example of integrity is the property that an attacker, who does not have authorized access to your bank account, cannot modify its contents. **Authenticity** is the state of knowing with a level of assurance the identity of agents you are communicating or interacting with. For instance, when you connect to a website that purports to be your bank, you want to be sure that it indeed is your bank's website and not that of some malicious entity. The process of demonstrating authenticity is known as **authentication**. Finally, **availability** is the property that a system provides a sufficient quality of service to its users. For example, you might expect your bank's website to be available to you 99% of the time.

It is important to note that security and privacy are *not* absolute properties. Do not believe anyone who claims that their system is "completely secure"! Security and privacy can only be guaranteed for a specific threat model and a specific set of properties. As a system designer, if security and privacy are important concerns for you, you must first

define your threat model and formalize your properties. Otherwise, any solution you adopt is essentially meaningless.

In this chapter, we seek to give you a basic understanding of security and privacy with a special focus on concepts relevant for cyber-physical system design. The field of security and privacy is too broad for us to cover it in any comprehensive manner in a single chapter; we refer the interested reader to some excellent textbooks on the topic, e.g., (Goodrich and Tamassia, 2011; Smith and Marchesini, 2007). Instead, our goal is more modest: to introduce you to the important basic concepts, and highlight the topics that are specific to, or especially relevant for, embedded, cyber-physical systems.

17.1 Cryptographic Primitives

The field of cryptography is one of the cornerstones of security and privacy. The word "cryptography" is derived from the Latin roots "crypt" meaning *secret* and "graphia" meaning *write*, and thus literally means "the study of secret writing."

We begin with a review of the basic **cryptographic primitives** for encryption and decryption, secure hashing, and authentication. Issues that are particularly relevant for the design and analysis of embedded and cyber-physical systems are highlighted. The reader is warned that examples given in this chapter, particularly those listing code, are given at a high level of abstraction, and omit details that are critical to developing secure cryptographic implementations. See books such as (Menezes et al., 1996; Ferguson et al., 2010) for a more in-depth treatment of the subject.

17.1.1 Encryption and Decryption

Encryption is the process of translating a message into an encoded form with the intent that an adversary cannot recover the former from the latter. The original message is typically referred to as the **plaintext**, and its encoded form as the **ciphertext**. **Decryption** is the process of recovering the plaintext from the ciphertext.

The typical approach to encryption relies on the presence of a secret called the **key**. An encryption algorithm uses the key and the plaintext in a prescribed manner to obtain the ciphertext. The key is shared between the parties that intend to exchange a message securely. Depending on the mode of sharing, there are two broad categories of cryptography.

In **symmetric-key cryptography**, the key is a single entity that is known to both sender and receiver. In **public-key cryptography**, also termed **asymmetric cryptography**, the key is split into two portions, a public part and a private part, where the **public key** is known to everyone (including the adversary) and the **private key** is known only to the receiver. In the rest of this section, we present a brief introduction to these two approaches to cryptography.

One of the fundamental tenets of cryptography, known as **Kerckhoff's principle**, states that a cryptographic system (algorithm) should be secure even if everything about the system, except the key, is public knowledge. In practical terms, this means that even if the adversary knows all details about the design and implementation of a cryptographic algorithm, as long as he does not know the key, he should be unable to recover the plaintext from the ciphertext.

Symmetric-Key Cryptography

Let Alice and Bob be two parties that seek to communicate with each other securely. In symmetric-key cryptography, they accomplish this using a shared secret key K. Suppose Alice wishes to encrypt and send Bob an n-bit plaintext message, $M = m_1 m_2 m_3 \dots m_n \in \{0,1\}^n$. We wish to have an encryption scheme that, given the shared key K, should encode M into ciphertext C with the following two properites. First, the intended recipient Bob should be able to easily recover M from C. Second, any adversary who does not know K should not, by observing C, be able to gain any more information about M.

We present intuition for the operation of symmetric-key cryptography using a simple, idealized scheme known as the **one-time pad**. In this scheme, the two parties Alice and Bob share an n-bit secret key $K = k_1 k_2 k_3 \dots k_n \in \{0,1\}^n$, where the n bits are chosen independently at random. K is known as the one-time pad. Given K, encryption works by taking the bit-wise XOR of M and K; i.e., $C = M \oplus K$ where $\oplus$ denotes XOR. Alice then sends C over to Bob, who decrypts C by taking the bit-wise XOR of C with K; using properties of the XOR operation, $C \oplus K = M$.

Suppose an adversary Eve observes C. We claim that Eve has no more information about M or K than she had without C. To see this, fix a plaintext message M. Then, every unique ciphertext $C \in \{0,1\}^n$ can be obtained from M with a corresponding unique choice of key K — simply set $K = C \oplus M$ where C is the desired ciphertext. Put another way, a uniformly random bit-string $K \in \{0,1\}^n$ generates a uniformly random ciphertext

$C \in \{0,1\}^n$. Thus, looking at this ciphertext, Eve can do no better than guessing at the value of K uniformly at random.

Notice that Alice cannot use the key K more than once! Consider what happens if she does so twice. Then Eve has access to two ciphertexts $C_1 = M_1 \oplus K$ and $C_2 = M_2 \oplus K$. If Eve computes $C_1 \oplus C_2$, using properties of XOR, she learns $M_1 \oplus M_2$. Thus, Eve receives partial information about the messages. In particular, if Eve happens to learn one of the messages, she learns the other one, and can also recover the key K. Thus, this scheme is not secure if the same key is used for multiple communications, hence the name "one-time pad." Fortunately, other, stronger symmetric key cryptography schemes exist.

Most common symmetric key encryption methods use a building block termed a **block cipher**. A **block cipher** is an encryption algorithm based on using a k-bit key K to convert an n-bit plaintext message M into an n-bit ciphertext C. It can be mathematically captured in terms of an encryption function $E : \{0,1\}^k \times \{0,1\}^n \to \{0,1\}^n$; i.e., $E(K, M) = C$. For a fixed key K, the function E_K defined by $E_K(M) = E(K, M)$ must be a permutation from $\{0,1\}^n$ to $\{0,1\}^n$. Decryption is the inverse function of encryption; note that the inverse exists for each K as E_K is invertible. We denote the decryption function corresponding to E_K by $D_K = E_K^{-1}$. Note that for simplicity this model of encryption abstracts it to be (only) a function of the message and the key; in practice, care must be taken to use a suitable block cipher mode that, for example, does not always encrypt the same plaintext to the same ciphertext.

One of the classic block ciphers is the **data encryption standard** (**DES**). Introduced in the mid-1970s, DES was the first block cipher based on modern cryptographic techniques, considered to be "commercial-grade," with an open specification. While the details of DES are beyond the scope of this chapter, we note that versions of DES are still used in certain embedded devices. For instance, a version of DES, known as **3DES**, which involves applying the DES algorithm to a block three times, is used in certain "chip and PIN" payment cards around in the world. The basic version of the DES involves use of a 56-bit key and operates on 64-bit message blocks. While DES initially provided an acceptable level of security, by the mid-1990s, it was getting easier to break it using "brute-force" methods. Therefore, in 2001, the U.S. National Institute of Standards and Technology (NIST) established a new cryptographic standard known as the **advanced encryption standard** or **AES**. This standard is based on a cryptographic scheme called *Rijndael* proposed by Joan Daemen and Vincent Rijmen, two researchers from Belgium. AES uses a message block length of 128 bits and three different key lengths of 128, 192, and 256 bits. Since its adoption, AES has proved to be a strong cryptographic algorithm amenable to efficient implementation both in hardware and in software. It is estimated

that the current fastest supercomputers could not successfully mount a brute-force attack on AES within the estimated lifetime of the solar system!

Public-Key Cryptography

In order to use symmetric-key cryptography, Alice and Bob need to have already set up a shared key in advance. This is not always easy to arrange. Public-key cryptography is designed to address this limitation.

In a public-key cryptosystem, each principal (Alice, Bob, etc.) has two keys: a *public key*, known to everyone, and a *private key*, known only to that principal. When Alice wants to send Bob a secret message, she obtains his public key K_B and encrypts her message with it. When Bob receives the message, he decrypts it with his private key k_B. In other words, the encryption function based on K_B must be invertible using the decryption function based on k_B, and moreover, must not be invertible without k_B.

Let us consider the last point. Encryption using a public key must effectively be a **one-way function**: a publicly-known function F such that it is easy to compute $F(M)$ but (virtually) impossible to invert without knowledge of a suitable secret (the private key). Remarkably, there is more than one public-key cryptographic scheme available today, each based on a clever combination of mathematics, algorithm design, and implementation tricks. We focus our attention here on one such scheme proposed in 1978 (Rivest et al.), known as **RSA** for the initials of its creators: Rivest, Shamir and Adleman. For brevity, we mostly limit our discussion to the basic mathematical concepts behind RSA.

Consider the setting where Bob wishes to create his public-private key pair so that others can send him secret messages. The RSA scheme involves three main steps, as detailed below:

1. *Key Generation:* Select two large prime numbers p and q, and compute their product $n = pq$. Then compute $\varphi(n)$ where φ denotes **Euler's totient function**. This function maps an integer n to the number of positive integers less than or equal to n that are relatively prime to n. For the special case of the product of primes, $\varphi(n) = (p-1)(q-1)$. Select a random integer e such that $1 < e < \varphi(n)$ such that the greatest common divisor of e and $\varphi(n)$, denoted $GCD(e, \varphi(n))$ equals 1. Then compute d, $1 < d < \varphi(n)$, such that $ed \equiv 1 \pmod{\varphi(n)}$. The key generation process is now complete. Bob's public key K_B is the pair (n, e). The private key is $k_B = d$.

2. *Encryption:* When Alice wishes to send a message to Bob, she first obtains his public key $K_B = (n, e)$. Given the message M to transmit, she computes the ciphertext C as $C = M^e \pmod{n}$. C is then transmitted to Bob.

3. *Decryption:* Upon receipt of C, Bob computes $C^d \pmod{n}$. By properties of Euler's totient function and modular arithmetic, this quantity equals M, allowing Bob to recover the plaintext message from Alice.

We make two observations on the above scheme. First, the operations in the various steps of RSA make heavy use of non-linear arithmetic on large numbers, especially modular multiplication of large integers and modular exponentiation. These operations must be implemented carefully, especially on embedded platforms, in order to operate efficiently. For instance, modular exponentiation is typically implemented using a version of the square-and-multiply technique outlined in Example 16.1 of Chapter 16. Second, the effectiveness of the approach depends critically on infrastructure to store and maintain public keys. Without such a **public-key infrastructure**, an attacker Eve can masquerade as Bob, advertising her own public key as Bob's, and thus fooling Alice into sending her messages intended for Bob. While such a public-key infrastructure has now been established for servers on the Web using so-called "certificate authorities," it is more challenging to extend that approach for networked embedded systems for a variety of reasons: the large scale of devices that may act as servers, the ad-hoc nature of networks, and the resource constraints on many embedded devices.[1] For this reason, public-key cryptography is not as widely used in embedded devices as is symmetric-key cryptography.

17.1.2 Digital Signatures and Secure Hash Functions

The cryptographic primitives for encryption and decryption help in providing confidentiality of data. However, by themselves, they are not designed to provide integrity or authenticity guarantees. Those properties require the use of related, but distinct, primitives: digital signatures, secure hash functions, and message authentication codes (MACs). This section provides a brief overview of these primitives.

[1] *Let's Encrypt*, an ongoing effort to develop a free, automated, and open certificate authority might mitigate some of these challenges; see `https://letsencrypt.org/` for more details.

Secure Hash Functions

A **secure hash function** (also known as *cryptographic hash function*) is a deterministic and keyless function $H : \{0,1\}^n \rightarrow \{0,1\}^k$ that maps n-bit messages to k-bit hash values. Typically, n can be arbitrarily large (i.e., the message can be arbitrarily long) while k is a relatively small fixed value. For example, SHA-256 is a secure hash function that

Implementation Matters for Cryptography

Embedded platforms typically have limited resources such as memory or energy, and must obey physical constraints, for instance on real-time behavior. Cryptography can be computationally quite demanding. Such computations cannot simply be offloaded "to the cloud" as secret data must be secured at the end point before being sent out on the network. Thus, cryptographic primitives must be implemented in a manner that obeys the constraints imposed by embedded platforms.

Consider, for example, public-key cryptography. These primitives involve modular exponentiation and multiplications that can have large timing, memory, and energy requirements. Consequently, researchers have developed efficient algorithms as well as software and hardware implementations. A good example is **Montgomery multiplication** (Montgomery, 1985), a clever way of performing modular multiplication by replacing the division (implicit in computing the modulus) with addition. Another promising technology is **elliptic curve cryptography** which allows for a level of security comparable with the RSA approach with keys of smaller bit-width, reducing the memory footprint of implementations; see, for example, Paar and Pelzl (2009) for details. Due to the costs of public-key cryptography, it is often used to exchange a shared symmetric key that is then used for all subsequent communication.

Another important aspect of implementing cryptographic schemes is the design and use of **random number generation** (**RNG**). A high-quality RNG requires as input a source of randomness with high entropy. Many recommended high entropy sources involve physical processes, such atmospheric noise or atomic decay, but these may not be easy to use in embedded systems, especially in hardware implementations. Alternatively, one can use on-chip thermal noise and the timing of input events from the physical world and network events. Polling the physical world for many such sources of randomness may consume power, leading to implementation trade-offs; see Perrig et al. (2002) for an example.

maps messages to a 256-bit hash, and is being adopted for authenticating some software packages and for hashing passwords in some Linux distributions. For a message M, we term $H(M)$ as the "hash of M."

A secure hash function H has certain important properties that distinguishes it from more traditional hash functions used for non-security purposes:

- *Efficient to compute:* Given a message M, it should be efficient to compute $H(M)$.
- *Pre-image resistance:* Given a hash (value) h, it should be computationally infeasible to find a message M such that $h = H(M)$.
- *Second pre-image resistance:* Given a message M_1, it should be computationally infeasible to find another message M_2 such that $H(M_1) = H(M_2)$. This property prevents attackers from taking a known message M_1 and modifying it to match a desired hash value (and hence find the corresponding message M_2).
- *Collision resistance:* It should be computationally infeasible to find two different messages M_1 and M_2 where $H(M_1) = H(M_2)$. This property, a stronger version of second pre-image resistance, prevents an attacker from taking *any* starting message and modifying it to match a given hash value.

A common use of secure hash functions is to verify the integrity of a message or a piece of data. For example, before you install a software update on your computer you might want to verify that the update you downloaded is a valid copy of the package you are updating and not something modified by an attacker. Providing a secure hash of the software update on a separate channel than the one on which the update file is distributed can provide such assurance. After separately downloading this hash value, you can verify, by computing the hash yourself on the software, that the provided hash indeed matches the one you compute. With the growing trend for embedded systems to be networked and to receive software patches over the network, the use of secure hash functions is expected to be a central component of any solution that maintains system integrity.

Digital Signatures

A **digital signature** is a cryptographic mechanism, based on public-key cryptography, for the author of a digital document (message) to authenticate himself/herself to a third party and to provide assurance about the integrity of the document. The use of digital signatures involves the following three stages:

1. *Key generation:* This step is identical to the key generation phase of public-key cryptosystems, resulting in a public key-private key pair.

2. *Signing:* In this step, the author/sender digitally signs the document to be sent to the receiving party.

3. *Verification:* The recipient of a signed document verifies, using the digital signature, that the sender of the document is indeed who he/she purports to be.

We now illustrate how the basic scheme works for the RSA cryptosystem. Suppose Bob wishes to digitally sign a message M before sending it to Alice. Recall from Section 17.1.1 that, using RSA, Bob can generate his public key as $K_B = (n, e)$ and private key $k_B = d$. One simple authentication scheme is for Bob to use his key-pair in reverse: to sign M, he simply encrypts it with his private key, computing $S = M^d \pmod{n}$, and sends S to Alice along with M. When Alice receives this signed message, she uses K_B to recover M from the signature S and verifies it by comparing it with the received message. If they match, the message has been authenticated; otherwise, Alice has detected tampering either in the message or its signature.

The above scheme is simple, but flawed. In particular, given the signatures S_1 and S_2 for two messages M_1 and M_2, notice that an attacker can construct the signature for message $M_1 \cdot M_2$ by simply multiplying together S_1 and $S_2 \pmod{n}$. In order to guard against this attack, one can first compute a secure hash of M_1 and M_2 before computing the signatures on the resultant hashes rather than on the messages.

Message Authentication Codes

A **message authentication code** (**MAC**) is a cryptographic mechanism, based on symmetric-key cryptography, for providing integrity and authenticity to a document. It is thus the symmetric-key analog of a digital signature. As in other symmetric-key schemes, the use of a MAC requires first setting up a shared key between sender and receiver that each of them can use to authenticate messages sent by the other. For this reason, MACs are best suited to settings where such a shared key can be easily set up. For instance, modern automobiles comprise multiple electronic control units (ECUs) communicating with each other on an on-board network such as a **controller-area network** (**CAN**) bus. In this case, if each ECU is pre-programmed with a common key, then each ECU can authenticate messages sent from other ECUs on the CAN bus using MACs.

17.2 Protocol and Network Security

An attacker typically gains access to a system via its connection to a network or some other medium used to connect it with other components. Additionally, it is increasingly the case that a single embedded system comprises many distributed components connected over a network. Thus, one must address fundamental questions about security over a network and using various communication protocols, a field termed **protocol security** or **network security**. In this section, we review basic ideas related to two topics of particular relevance for embedded systems: *key exchange* and *cryptographic protocol design*.

17.2.1 Key Exchange

We have already seen how secret keys are critical components of both symmetric and asymmetric cryptosystems. How are these keys established in the first place? In a symmetric cryptosystem, we need a mechanism for two communicating parties to agree on a single shared key. In an asymmetric cryptosystem, we need infrastructure for establishing and maintaining public keys so that any party on a network can look up the public key of any other party it wishes to communicate with. In this section, we discuss a classic method for **key exchange** and some alternative schemes customized to specific embedded system design problems, with a focus on symmetric cryptography.

Diffie-Hellman Key Exchange

In 1976, Whitfield Diffie and Martin Hellman introduced what is widely considered the first public-key cryptosystem (Diffie and Hellman, 1976). The crux of their proposal was a clever scheme for two parties to agree on a shared secret key using a communication medium observable by anyone. This scheme is commonly referred to as **Diffie-Hellman** key exchange.

Suppose there are two parties Alice and Bob that wish to agree on a secret key. Everything Alice sends to Bob, and vice-versa, can be viewed by an attacker Eve. Alice and Bob wish to agree on a key while ensuring that it is computationally infeasible for Eve to compute that key by observing their communication. Here is how Alice and Bob can use Diffie-Hellman key exchange to achieve this objective.

To begin with, Alice and Bob need to agree on two parameters. In doing so, they can use various methods including a hard-coded scheme (e.g., a parameter hard-coded into the same program they use) or one of them can announce the parameters to the other in some deterministic fashion (e.g., based on a fixed ordering between their network addresses). The first parameter is a very large prime number p. The second is a number z such that $1 < z < p - 1$. Note that an attacker can observe z and p.

After these parameters p and z are agreed upon, Alice randomly selects a number a from the set $\{0, 1, \ldots, p - 2\}$ and keeps it secret. Similarly, Bob randomly selects a number $b \in \{0, 1, \ldots, p - 2\}$ and keeps it secret. Alice computes the quantity $A = z^a \pmod p$ and sends it to Bob. Likewise, Bob computes $B = z^b \pmod p$ and sends it to Alice. In addition to z and p, the attacker Eve can observe A and B, but not a and b.

On receiving B, Alice uses her secret number a to compute $B^a \pmod p$. Bob performs the analogous step, computing $A^b \pmod p$. Now, observe that

$$A^b \pmod p = z^{ab} \pmod p = B^a \pmod p$$

Thus, amazingly, Alice and Bob have agreed on a shared key $K = z^{ab} \pmod p$ simply by communicating on a public channel. Note that they have revealed neither of the secret numbers a or b to the attacker Eve. Without knowledge of a or b, Eve cannot reliably compute K. Moreover, it is computationally very difficult for Eve to compute a or b simply by knowing p, z, A or B, since the underlying problem is one known as the **discrete logarithm** problem, for which no efficient (polynomial-time) algorithm is known. Put another way, the function $f(x) = z^x \pmod p$ is a one-way function.

Diffie-Hellman is simple, elegant, and effective: unfortunately, it is typically not practical for resource-constrained embedded systems. Computing it involves modular exponentiation for large primes (typical key sizes range to 2048-bits), and thus impractical for energy-constrained platforms that must obey real-time constraints. We therefore discuss next a few alternative schemes.

Timed Release of Keys

Many networked embedded systems use a **broadcast** medium for communication — one where senders send data over a public channel where all receivers, intended and unintended, can read that data. Examples include on-board automotive networks using the CAN bus and wireless sensor networks. A broadcast medium has many advantages: it is simple, requires little infrastructure, and a sender can quickly reach a large number of

receivers. However, it also has certain disadvantages, notably that malicious parties can eavesdrop and inject malicious packets with ease. Reliability of the broadcast medium can also be concern.

How can one achieve secure key exchange in such a broadcast medium, under constraints on timing and energy consumption? This question was studied in the early 2000s, when the first work was done on deploying wireless sensor networks (e.g., see Perrig et al. (2004); Perrig and Tygar (2012)). One of the novel ideas is to leverage *timing properties* of these networks, an idea whose roots go back to a paper by Anderson et al. (1998).

The first property is that of **clock synchronization**, where different nodes on the network have clocks such that the difference between the values of any two clocks is bounded by a small constant. Many sensor nodes can have synchronized clocks via GPS or protocols such as the **precision time protocol** (**PTP**) (Eidson, 2006).

The second property is that of *scheduled transmission*. Each node in the network transmits packets at pre-determined times, following a schedule.

The combination of the above two properties permits the use of a secure broadcast protocol called μTESLA (Perrig et al., 2002). This protocol is designed for broadcast networks comprising two types of nodes, *base stations* and *sensor nodes*. A key property of μTESLA is *secure authentication* — a node receiving a message should be able to determine who the authentic sender is, discarding spoofed messages. It performs this by using a message authentication code (MAC) in each message, which is computed using a secret key. This secret key is broadcast some time after the original message was sent, allowing the receiver to compute the MAC for itself, and determine the authenticity of the received message. Since each message is timestamped, upon receiving a message the receiver verifies that the key used to compute the MAC has not yet been disclosed based on this timestamp, its local clock, the maximum clock skew, and the time schedule at which keys are disclosed. If the check fails, the message is discarded; otherwise, when the key is received, the authenticity of the message is checked by computing the MAC and checking against the received MAC. In this way, the *timed release* of keys is used to share keys between a broadcast sender and its receivers, for the purpose of authenticating messages.

A similar approach has been adopted for CAN-based automotive networks. Lin et al. (2013) have shown how to extend the μTESLA approach for the CAN bus where the nodes do not necessarily share a common global notion of time. Timed delayed release

of keys has also been implemented for Time Triggered Ethernet (TTE) (Wasicek et al., 2011).

Other Schemes

In certain application domains, customized schemes have been developed for key exchange using special characteristics of those domains. For instance, Halperin et al. (2008) discuss a key exchange scheme for **implantable medical devices** (**IMD**s) such as pacemakers and defibrillators. They propose a "zero-power" security mechanism on the implanted device based on radio-frequency (RF) power harvesting. The novel key exchange scheme involves a "programmer" device initiating communication with the IMD (and supplying an RF signal to power it). The IMD generates a random value that is communicated as a weak modulated sound wave that is perceptible by the patient. The security

Reverse Engineering Systems to Improve Security

Several embedded systems were designed at a time when security was a secondary design concern, if one at all. In recent years, researchers have shown how this lack of attention to security can lead to compromised systems in various domains. The chief approach taken is to reverse engineer protocols that provide access to systems and the working of their components.

We present here three representative examples of vulnerabilities demonstrated in real embedded systems. Halperin et al. (2008) show how implantable medical devices (IMDs) can be read and reprogrammed wirelessly. Koscher et al. (2010) demonstrate for automobiles how the on-board diagnostics bus (OBD-II) protocol can be subverted to adversarially control a wide range of functions, including disabling the brakes and stopping the engine. Ghena et al. (2014) study traffic light intersections and show how their functioning can be partially controlled wirelessly to perform, for example, a denial-of-service attack causing traffic congestion.

Such investigations based on reverse engineering "legacy" embedded systems to understand their functioning and uncover potential security flaws form an important part of the process of securing our cyber-physical infrastructure.

of this scheme relies on a threat model based on physical security around the patient's immediate vicinity, one that is plausible for this particular application.

17.2.2 Cryptographic Protocol Design

Some of the most-publicized vulnerabilities in networked embedded systems, at the time of writing, arise from the fact that the communication protocols were designed with little or no security in mind (see the box on page 472). However, even protocols designed for security using cryptographic primitives can have vulnerabilities. Poor protocol design can compromise confidentiality, integrity, and authenticity properties even if one assumes perfect cryptography and the presence of key exchange and key management infrastructure. We explain this point with an example.

Example 17.1: A **replay attack** on a protocol is one where an attacker is able to violate a security property by replaying certain messages in the protocol, possibly with modifications to accompanying data.

We describe here a replay attack on a fictitious wireless sensor network protocol. This network comprises a base station S and several energy-constrained sensor nodes $N_1, N_2, \ldots, N_k$ communicating via broadcast. Each node spends most of its time in a sleep mode, to conserve energy. When S wants to communicate with a particular node N_i to take a sensor reading, it sends it a message M encrypted with a suitable shared key K_i pre-arranged between S and N_i. N_i then responds with an encrypted sensor reading R. In other words, the protocol has a message exchange as follows:

$$\begin{aligned} S &\rightarrow N_i : E(K_i, M) \\ N_i &\rightarrow S : E(K_i, R) \end{aligned}$$

This protocol, on the face of it, seems secure. However, it has a flaw that can hinder operation of the network. Since the nodes use broadcast communication, an attacker can easily record the message $E(K_i, M)$. The attacker can then replay this message multiple times at a low power so that it is detected by N_i, but not by S. N_i will keep responding with encrypted sensor readings, possibly causing it to run down its battery much faster than anticipated, and disabling its operation.

Notice that there was nothing wrong with the individual cryptographic steps used in this protocol. The problem was with the temporal behavior of the system —

that N_i had no way of checking if a message is "fresh." A simple countermeasure to this attack is to attach a fresh random value, called a **nonce**, or a timestamp with each message.

This attack is also an example of a so-called **denial-of-service attack**, abbreviated **DoS attack**, indicating that the impact of the attack is a loss of service provided by the sensor node.

Fortunately, techniques and tools exist to formally model cryptographic protocols and verify their correctness with respect to specific properties and threat models before deploying them. More widespread use of these techniques is needed to improve protocol security.

17.3 Software Security

The field of **software security** is concerned with how errors in the software implementation can impact desired security and privacy properties of the system. Bugs in software can and have been used to steal data, crash a system, and worse, allow an attacker to obtain an arbitrary level of control over a system. Vulnerabilities in software implementations are one of the largest categories of security problems in practice. These vulnerabilities can be especially dangerous for embedded systems. Much of embedded software is programmed in low-level languages such as C, where the programmer can write arbitrary code with few language-level checks. Moreover, the software often runs "bare metal" without an underlying operating system or other software layer that can monitor and trap illegal accesses or operations. Finally, we note that in embedded systems, even "crashing" the system can have severe consequences, since it can completely impair the interaction of the system with the physical world. Consider, for example, a medical device such as a pacemaker, where the consequences of a crash may be that the device stops functioning (e.g. pacing), leading to potentially life-threatening consequences for the patient.

In this section, we give the reader a very brief introduction to software security problems using a few illustrative examples. The reader is referred to relevant books on security (e.g., Smith and Marchesini (2007)) for further details.

Our examples illustrate a vulnerability known as a **buffer overflow** or **buffer overrun**. This class of errors is particularly common in low-level languages such as C. It arises from

the absence of automatic bounds-checking for accessing arrays or pointers in C programs. More precisely, a buffer overflow is a error arising from the absence of a bounds check resulting in the program writing past the end of an array or memory region. Attackers can use this out-of-bounds write to corrupt trusted locations in a program such as the value of a secret variable or the return address of a function. (Tip: material in Chapter 9 on Memory Architectures might be worth reviewing.) Let us begin with a simple example.

Example 17.2: The sensors in certain embedded systems use communication protocols where data from various on-board sensors is read as a stream of bytes from a designated port or network socket. The code example below illustrates one such scenario. Here, the programmer expects to read at most 16 bytes of sensor data, storing them into the array `sensor_data`.

```
1  char sensor_data[16];
2  int secret_key;
3
4  void read_sensor_data() {
5    int i = 0;
6
7    // more_data returns 1 if there is more data,
8    // and 0 otherwise
9    while(more_data()) {
10     sensor_data[i] = get_next_byte();
11     i++;
12   }
13
14   return;
15 }
```

The problem with this code is that it implicitly trusts the sensor stream to be no more than 16 bytes long. Suppose an attacker has control of that stream, either through physical access to the sensors or over the network. Then an attacker can provide more than 16 bytes and cause the program to write past the end of the array `sensor_data`. Notice further how the variable `secret_key` is defined right after `sensor_data`, and assume that the compiler allocates them adjacently. In this case, an attacker can exploit the buffer oveflow vulnerability to provide a stream of length 20 bytes and overwrite `secret_key` with a key of his choosing. This exploit can then be used to compromise the system in other ways.

The example above involves an out-of-bounds write in an array stored in global memory. Consider next the case when the array is stored on the stack. In this case, a buffer overrun vulnerability can be exploited to overwrite the return address of the function, and cause it to execute some code of the attacker's choosing. This can lead to the attacker gaining an arbitrary level of control over the embedded system.

Example 17.3: Consider below a variant on the code in Example 17.2 where the `sensor_data` array is stored on the stack. As before, the function reads a stream of bytes and stores them into `sensor_data`. However, in this case, the read sensor data is then processed within this function and used to set certain globally-stored flags (which can then be used to take control decisions).

```
1  int sensor_flags[4];
2
3  void process_sensor_data() {
4    int i = 0;
5    char sensor_data[16];
6
7    // more_data returns 1 if there is more data,
8    // and 0 otherwise
9    while(more_data()) {
10     sensor_data[i] = get_next_byte();
11     i++;
12   }
13
14   // some code here that sets sensor_flags
15   // based on the values in sensor_data
16
17   return;
18 }
```

Recall from Chapter 9 how the stack frame of a function is laid out. It is possible for the return address of function `process_sensor_data` to be stored in memory right after the end of the array `sensor_data`. Thus, an attacker can exploit the buffer overflow vulnerability to overwrite the return address and cause execution to jump to a location of his choice. This version of the buffer overflow exploit is sometimes (and rather aptly) termed as **stack smashing**.

Moreover, the attacker could write a longer sequence of bytes to memory and include arbitrary code in that sequence. The overwritten return address could be

tailored to be *within* this overwritten memory region, thus causing the attacker to control the code that gets executed! This attack is often referred to as a **code injection attack**.

How do we avoid buffer overflow attacks? One easy way is to explicitly check that we never write past the end of the buffer, by keeping track of its length. Another approach is to use higher-level languages that enforce **memory safety** — preventing the program from reading or writing to locations in memory that it does not have privileges for or that the programmer did not intend it to. Exercise 1 gives you some practice with writing code so as to avoid a buffer overflow.

17.4 Information Flow

Many security properties specify restrictions on the flow of information between principals. Confidentiality properties restrict the flow of secret data to channels that are readable by an attacker. For example, your bank balance should be viewable only by you and authorized bank personnel and not by an attacker. Similarly, integrity properties restrict the flow of untrusted values, controlled by an attacker, to trusted channels or locations. Your bank balance should be writable only by trusted deposit or withdrawal operations and not arbitrarily by a malicious party. Thus, **secure information flow**, the systematic study of how the flow of information in a system affects its security and privacy, is a central topic in the literature.

Given a component of an embedded system it is important to understand how information flows from secret locations to attacker-readable "public" locations, or from attacker-controlled untrusted channels to trusted locations. We need techniques to detect illegal information flows and to quantify their impact on the overall security of the system. Additionally, given security and privacy policies, we need techniques to enforce those policies on a system by suitably restricting the flow of information. Our goal, in this section, is to equip the reader with the basic principles of secure information flow so as to be able to develop such techniques.

17.4.1 Examples

In order to motivate the secure information flow problem, we present a series of examples. Although these examples focus on confidentiality, the same approaches and concerns apply for integrity properties as well.

Example 17.4: Medical devices are increasingly software-controlled, personalized, and networked. An example of such a device is a blood glucose meter, used by doctors and patients to monitor a patient's blood glucose level. Consider a hypothetical glucose meter that can take a reading of a patient's glucose level, show it on the device's display, and also transmit it over the network to the patient's hospital. Here is a highly-abstracted version of the software that might perform these tasks.

```
1  int patient_id; // initialized to the
2                  // patient's unique identifier
3  void take_reading() {
4    float reading = read_from_sensor();
5
6    display(reading);
7
8    send(network_socket, hospital_server,
9         reading, patient_id);
10
11   return;
12 }
```

The function `take_reading` records a single blood glucose reading (e.g., in mg/dL) and stores it in a floating-point variable `reading`. It then writes this value, suitably formatted, to the device's display. Finally, it transmits the value along with the patient's ID without any encryption over the network to the hospital server for analysis by the patient's doctor.

Suppose that the patient wishes to have his glucose level kept private, in the sense that its value is known only to him and his doctor, but no one else. Does this program achieve that objective? It is easy to see that it does not, as the value of `reading` is transmitted "in the clear" over the network. We can formalize this violation of privacy as an illegal information flow from `reading` to `network_socket`, where the former is a secret location whereas the latter is a public channel visible to an attacker.

With the knowledge of cryptographic primitives presented earlier in this chapter, we know that we can do better. In particular, let us assume the presence of a shared key between the patient's device and the hospital server that permits the use of symmetric-key encryption. Consider the following new version of the above example.

Example 17.5: In this version of the program, we employ symmetric-key cryptography, denoted by `AES`, to protect the reading. The encryption process is abstractly represented by the function `enc_AES`, and `send_enc` differs from `send` only in the type of its third argument.

```
1  int patient_id; // initialized to the
2                  // patient's unique identifier
3  long cipher_text;
4
5  struct secret_key_s {
6    long key_part1; long key_part2;
7  }; // struct type storing 128-bit AES key
8
9  struct secret_key_s secret_key; // shared key
10
11 void take_reading() {
12   float reading = read_from_sensor();
13
14   display(reading);
15
16   enc_AES(&secret_key, reading, &cipher_text);
17
18   send_enc(network_socket, hospital_server,
19            cipher_text, patient_id);
20
21   return;
22 }
```

In this case, there is still a flow of information from `reading` to `network_socket`. In fact, there is also a flow from `secret_key` to `network_socket`. However, both these flows passed through an encryption function denoted by `enc_AES`. In the security literature, such a function is termed a **declassifier** since it encodes the secret data in such a way that it becomes acceptable to transmit the encoded, "declassified" result to an attacker-readable channel. In other words, the declassifier serves as a "dam" blocking the flow of secret information and releasing only information that, by the properties of the cryptographic

primitives, provides attackers with no more information than they had before the release.

While the above fix to the code blocks the flow of information about the reading of the patient's blood glucose, note that it does not completely protect the program. In particular, notice that the patient's ID is still being sent in the clear over the network! Thus, even though the attacker cannot tell, without the secret key, what the patient's reading is, he/she knows who transmitted a reading to the hospital (and possibly also at what time). Some private data is still being leaked. One solution to this leak is fairly obvious – we can encrypt both variables `reading` and `patient_id` using the secret key before sending them over the network.

So far we have assumed an implicit threat model where the attacker can only read information sent over the network. However, for many embedded systems, attackers may also have physical access to the device. For instance, consider what happens if the patient loses his device and an unauthorized person (attacker) gains access to it. If the patient's readings are stored on the device, the attacker might be able to read the data stored on it if such an illegal information flow exists. One approach to guard against this attack is to have the patient create a password, much like one creates a password for other personal devices, and have the device prompt one for the password when turned on. This idea is discussed in the example below.

Example 17.6: Suppose that a log of the past 100 readings is stored on the device. The code below sketches an implementation of a function `show_readings` that prompts the user for an integer password, and displays the stored readings only if the current password stored in `patient_pwd` is entered, otherwise displaying an error message that has no information about the stored readings.

```
1 int patient_id; // initialized to the
2                 // patient's unique identifier
3 int patient_pwd; // stored patient password
4
5 float stored_readings[100];
6
7 void show_readings() {
```